```
application-test.yml      // 测试环境配置文件
application-pro.yml       // 生产环境配置文件
```

如果想要使用上述对应环境的配置文件，只需要在 Spring Boot 全局配置文件中激活指定的 Profile 即可。

下面通过案例演示在多文件中配置 Profile，具体如下。

（1）拆分配置文件。将文件 2-11 中开发环境、测试环境、生产环境的 Profile 拆分为 3 个文件，具体如文件 2-12～文件 2-14 所示。

文件 2-12　application-dev.yml

```
server:
  port: 80
```

文件 2-13　application-test.yml

```
server:
  port: 81
```

文件 2-14　application-pro.yml

```
server:
  port: 82
```

（2）测试程序效果。此时项目 application.yml 配置文件中指定激活的 Profile 为 pro，启动项目，控制台输出如图 2-10 所示。

从图 2-10 可以看出，控制台输出信息中提示项目启动时激活的 Profile 为 pro，Tomcat 的端口号为 82。

在浏览器中访问 http://localhost:82/getPort，效果如图 2-11 所示。

图2-10　控制台输出（3）

图2-11　访问效果（2）

从图 2-11 可以看出，可以通过 82 端口号访问项目，说明启动的环境为生产环境。

如果将 application.yml 中激活的 Profile 修改为 test，再次启动项目时，项目控制台输出如图 2-12 所示。

图2-12　控制台输出（4）

从图 2-12 可以看出，控制台输出信息中提示启动时激活的 Profile 为 test，Tomcat 的端口号为 81，说明启动项目时使用的是测试环境的配置信息。

至此，在多文件中配置 Profile 已经完成。

2.4.3 @Profile 注解

在默认情况下，项目启动后，所有的 Bean 在任何环境下都可以生效，如果想要指定某个 Bean 只在特定的配置环境下生效，可以使用@Profile 注解实现。@Profile 可以标注在类上和方法上。标注在类上时，通常类的上方需要被@Component 标注，以指定创建的 Bean 的生效环境；标注在方法上时，通常为配置类中被@Bean 标注的方法，以指定返回的 Bean 的生效环境。

下面通过案例演示@Profile 注解进行多环境配置的使用方法，具体如下。

（1）创建数据库连接接口。在 chapter02 项目的 com.itheima.config 包下，创建数据库配置的接口 DBConnector ,并在该接口中声明一个数据库配置连接方法 ,具体如文件 2-15 所示。

文件 2-15 DBConnector.java

```
1 public interface DBConnector {
2     public void configure();
3 }
```

（2）创建数据库连接实现类。在 chapter02 项目的 com.itheima.config 包下，根据创建的数据库连接接口 DBConnector ,创建不同环境的数据库连接实现类 DevDBConnector 和 ProDBConnector，并重写 configure()方法模拟进行不同数据库环境的连接配置，具体如文件 2-16 和文件 2-17 所示。

文件 2-16 DevDBConnector.java

```
1 import org.springframework.stereotype.Component;
2 import org.springframework.context.annotation.Profile;
3 @Component
4 @Profile("dev")    // 指定多环境配置类标识
5 public class DevDBConnector implements DBConnector {
6     @Override
7     public void configure() {
8         System.out.println("数据库配置环境 dev");
9     }
10 }
```

文件 2-17 ProdDBConnector.java

```
1 import org.springframework.stereotype.Component;
2 import org.springframework.context.annotation.Profile;
3 @Component
4 @Profile("pro")   // 指定多环境配置类标识
5 public class ProDBConnector implements DBConnector {
6     @Override
7     public void configure() {
8         System.out.println("数据库配置环境 prod");
9     }
10 }
```

在文件 2-16 和文件 2-17 中，类上使用@Component 注解进行声明，以确保 Spring Boot 启动时扫描并加载类对象到 Spring 容器中，同时使用@Profile 注解指定 Bean 的生效环境。在具体的数据库连接配置方法 configure()中 ,文件 2-16 和文件 2-17 中只是做了简单的输出语句打印操作，具体配置可以根据实际开发需求来编写。

（3）新增控制器方法。在文件 2-11 的 DefaultController 类中注入 DBConnector 对象，并新增方法 showDB()，在该方法中执行数据库连接配置方法，具体代码如下。

```
@Autowired
private DBConnector connector;
@RequestMapping("/showDB")
public void showDB() {
    connector.configure();
}
```

（4）测试程序效果。在项目 application.yml 配置文件中指定激活的 Profile 为 pro，启动项目，控制台输出如图 2-13 所示。

从图 2-13 可以看出，控制台输出信息中提示项目启动时激活的 Profile 为 pro，Tomcat 的端口号为 82。

在浏览器中访问 http://localhost:82/showDB，控制台输出如图 2-14 所示。

从图 2-14 可以看出，控制台输出的为 pro 数据库配置环境，说明自动装配时注入的具体对象为 ProDBConnector 对象，@Profile 注解可以根据指定的环境注入符合当前运行环境的相应的 Bean。

图2-13 控制台输出（5）

图2-14 控制台输出（6）

2.5 本章小结

本章主要对 Spring Boot 配置进行了讲解。首先讲解了全局配置文件；然后讲解了配置绑定；接着讲解了引入配置文件和定义配置类；最后讲解了 Profile。通过本章的学习，希望大家可以掌握 Spring Boot 的基本配置方法，为后续更深入地学习 Spring Boot 做好铺垫。

2.6 本章习题

一、填空题

1．application.properties 文件中属性通过_____格式进行设置。

2．application.yml 中配置的属性为数组类型或单列集合时，“ - ”和属性值之间使用_____间隔。

3．@ConfigurationProperties 注解的_____属性可以指定绑定配置文件中属性的前缀。

4．_____可以让 Spring Boot 针对不同的环境提供不同的配置。

5．@ConfigurationProperties 注解的_____属性可以指定绑定配置文件中属性的前缀。

二、判断题

1．在默认情况下，application.properties 文件中配置的字符串必须使用单引号或双引号

进行修饰。(　　)

2．YAML 配置文件的行内式写法配置单列集合属性，包含属性值的中括号“[]”可以省略。(　　)

3．@Value 注解支持以 SpEL 表达式进行 Bean 的属性值注入。(　　)

4．@ConfigurationProperties 注解进行配置文件属性注入时，不需要设置属性的 setter 方法。(　　)

5．在 Spring Boot 中可以将 Profile 配置在多个文件中。(　　)

三、选择题

1．下列选项中，关于 application.yml 中配置属性的规则描述错误的是(　　)。

A．使用缩进表示层级关系。　　B．可以使用“Tab”键进行缩进。

C．同级元素必须左侧对齐。　　D．大小写敏感。

2．下列选项中，对@Value 注解的描述错误的是(　　)。

A．Spring Boot 框架从 Spring 框架中对@Value 注解进行了默认继承。

B．@Value 可以将配置文件中的属性绑定到 Bean 对象对应的属性。

C．使用@Value 注解对属性注入值时，类中必须同时提供属性的 getter()和 setter()方法。

D．@Value 注解对 Map 集合的属性注入支持效果不佳。

3．下列选项中，关于@ConfigurationProperties 注解的描述错误的是(　　)。

A．@ConfigurationProperties 注解可以将配置文件中的一组配置数据同时绑定到 Bean 中。

B．@ConfigurationProperties 注解的 prefix 属性用于指定绑定配置文件中属性的前缀。

C．实体类中的属性名需要与绑定的配置文件中的属性名保持一致。

D．@ConfigurationProperties 注解时，也需要在属性上方标注@Value 注解。

4．下列选项中，关于在 Spring Boot 的配置文件中配置 Profile 的描述错误的是(　　)。

A．可以在配置文件使用 spring.config.activate.on-profile 指定 Profile 的名称。

B．使用 spring.profiles.active 指定激活哪个 Profile。

C．如果需要激活多个 Profile，Profile 名称之间使用逗号间隔即可。

D．单一 YAML 文件中只能配置单个 Profile。

5．下列选项中，在 Spring Boot 配置文件中用于激活指定 Profile 的属性是(　　)。

A．spring.config.activate.on-profile　　B．spring.profiles.active

C．server.port　　D．spring.application.name

第3章

Spring Boot 的Web应用支持

学习目标

★ 了解使用 Spring Bean 注册 Java Web 三大组件，能够简述使用 Spring Bean 注册 Java Web 三大组件的步骤

★ 了解使用 RegistrationBean 注册 Java Web 三大组件，能够简述使用 RegistrationBean 注册 Java Web 三大组件的步骤

★ 了解使用注解扫描注册 Java Web 三大组件，能够简述使用注解扫描注册 Java Web 三大组件的步骤

★ 了解 Spring MVC 自动配置的特性，能够说出 Spring MVC 自动配置的特性

★ 掌握自定义 Spring MVC 配置，能够自定义配置 Spring MVC 中的静态资源映射、视图控制器、拦截器

★ 掌握文件上传，能够在 Spring Boot 项目中实现文件上传

★ 熟悉 Spring Boot 异常处理自动配置原理，能够说出 Spring Boot 异常处理自动配置原理

★ 掌握 Spring Boot 自定义异常处理，能够在 Spring Boot 项目中自定义异常处理

通常在 Web 开发中，会涉及静态资源的访问支持、视图解析器的配置、转换器和格式化器的定制、文件上传等功能，甚至还需要考虑与 Web 服务器关联的 Java Web 三大组件的定制。Spring Boot 框架支持整合一些常用 Web 框架来实现 Web 开发，并默认支持 Web 开发中的一些通用功能。下面将对 Spring Boot 的 Web 应用支持进行讲解。

3.1 注册 Java Web 三大组件

在传统 Java Web 应用开发中，最常用的三大组件有 Servlet、Filter 和 Listener，开发者使用这些组件时需要在项目的 web.xml 文件中进行配置，或者使用相应的注解进行标注。Spring Boot 项目中默认使用内嵌的 Servlet 容器，项目中默认没有 web.xml 文件，同时默认情况下 Spring Boot 项目不能自动识别到这三个组件的相关注解。但 Spring Boot 提供了其他注册 Servlet、Filter 和 Listener 的方法，开发者可以使用 Spring Bean、RegistrationBean、注解扫描的方式注册 Java Web 三大组件。下面分别对这三种注册 Java Web 三大组件的方式进行讲解。

3.1.1　使用 Spring Bean 注册 Java Web 三大组件

在 Spring Boot 项目中，会自动将 Spring 容器中的 Servlet、Filter、Listener 实例注册为 Web 服务器中对应的组件。因此，可以将自定义的 Java Web 三大组件作为 Bean 添加到 Spring 容器中，以实现组件的注册。

使用 Spring Bean 注册 Servlet 时，需要自定义两个及以上的 Servlet，Servlet 对应的映射地址为“Bean 名称+/”，Filter 的映射地址默认为“/*”。下面通过案例演示使用 Spring Bean 注册 Java Web 三大组件，具体如下。

1. 创建自定义原生组件

创建 Spring Boot 项目 chapter03，在项目的 java 文件夹下创建类包 com.itheima.chapter03.web，并在类包下创建自定义的 Servlet、Filter、Listener，具体如文件 3-1～文件 3-4 所示。

文件 3-1　FirstServlet.java

```
1  import javax.servlet.http.HttpServlet;
2  import javax.servlet.http.HttpServletRequest;
3  import javax.servlet.http.HttpServletResponse;
4  import java.io.IOException;
5  public class FirstServlet extends HttpServlet {
6      @Override
7      public void doGet(HttpServletRequest req, HttpServletResponse resp)
8          throws  IOException {
9          System.out.println("hello FirstServlet");
10         resp.getWriter().write("hello FirstServlet");
11     }
12 }
```

在上述代码中，FirstServlet 类通过继承 HttpServlet 创建为 Servlet。

文件 3-2　SecondServlet.java

```
1  import javax.servlet.http.HttpServlet;
2  import javax.servlet.http.HttpServletRequest;
3  import javax.servlet.http.HttpServletResponse;
4  import java.io.IOException;
5  public class SecondServlet extends HttpServlet {
6      @Override
7      public void doGet(HttpServletRequest req, HttpServletResponse resp)
8          throws  IOException {
9          System.out.println("hello SecondServlet");
10         resp.getWriter().write("hello SecondServlet");
11     }
12 }
```

文件 3-3　MyFilter.java

```
1  import javax.servlet.*;
2  import java.io.IOException;
3  public class MyFilter  implements Filter {
4      @Override
5      public void doFilter(ServletRequest requ, ServletResponse resp,
6          FilterChain filterChain) throws IOException, ServletException {
7          System.out.println("处理请求前的处理");
```

```
        filterChain.doFilter(requ,resp);
        System.out.println("处理请求后的处理");
    }
}
```

在上述代码中，MyFilter 类通过实现 Filter 接口创建为 Filter。

文件 3-4 MyListener.java

```
import javax.servlet.ServletContextEvent;
import javax.servlet.ServletContextListener;
public class MyListener implements ServletContextListener {
    @Override
    public void contextInitialized(ServletContextEvent sce) {
        System.out.println("----Web 应用初始化完成----");
    }
    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        System.out.println("----Web 应用销毁之前----");
    }
}
```

Servlet 规范提供了很多监听器接口供开发者使用，分别用于实现监听不同目标的监听器。在上述代码中 MyListener 类通过实现 ServletContextListener 接口，可以监听 Web 应用初始化和应用销毁。

2. 创建组件配置类

在项目的 java 文件夹下创建类包 com.itheima.chapter03.config，并在类包下创建组件配置类 WebConfigure，在该类中创建 4 个方法，然后在这 4 个方法中分别创建文件 3-1～文件 3-4 中对应类的实例，并将这些创建的实例交由 Spring 管理，具体如文件 3-5 所示。

文件 3-5 WebConfigure.java

```
import com.itheima.chapter03.web.FirstServlet;
import com.itheima.chapter03.web.MyFilter;
import com.itheima.chapter03.web.MyListener;
import com.itheima.chapter03.web.SecondServlet;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
@Configuration
public class WebConfigure {
    @Bean("firstServlet")
    public FirstServlet firstServlet(){
        return  new FirstServlet();
    }
    @Bean("secondServlet")
    public SecondServlet secondServlet(){
        return  new SecondServlet();
    }
    @Bean
    public MyFilter myFilter(){
        return  new MyFilter();
    }
    @Bean
    public MyListener myListener(){
```

```
23        return  new MyListener();
24    }
25 }
```

在上述代码中，第 7 行代码使用@Configuration 注解标注当前类为注解类；第 9～16 行代码分别创建了 FirstServlet 实例和 SecondServlet 实例，通过@Bean 注解将创建的实例交由 Spring 管理，并定义 Bean 的名称为 firstServlet 和 secondServlet。

3. 测试程序效果

运行项目的启动类，控制台输出如图 3-1 所示。

图3-1　控制台输出（1）

从图 3-1 可以看出，控制台输出 Web 应用初始化完成等信息，说明项目启动时，Web 服务器中成功注册了 MyListener ,在监听到 Web 应用初始化时 ,执行了对应的方法进行输出。

在浏览器中访问 http://localhost:8080/firstServlet/，此时控制台输出如图 3-2 所示。

图3-2　控制台输出（2）

从图 3-2 可知，在浏览器中访问 FirstServlet 时，项目中的拦截器拦截到对应的请求，在放行后执行了 FirstServlet 的 doGet()方法，说明项目启动时，Web 服务器中成功注册了 FirstServlet 和 MyFilter。

3.1.2　使用 RegistrationBean 注册 Java Web 三大组件

使用 Spring Bean 注册 Java Web 三大组件时，如果容器中只有一个自定义 Servlet，则无法使用 Bean 的名称作为映射路径，而 Filter 默认只使用“ /* ”的映射地址。为解决此问题，Spring Boot 提供了更为灵活的注册方法，可以在配置类中使用 RegistrationBean 来注册原生 Web 组件。

RegistrationBean 是个抽象类，SpringBoot 提供了三个 RegistrationBean 的实现类：ServletRegistrationBean、FilterRegistrationBean、ServletListenerRegistrationBean，这三个类分别用来注册 Servlet、Filter 和 Listener，通过这三个类开发者可以获得自定义映射路径等更多的控制权。

下面通过案例演示使用 RegistrationBean 注册 Java Web 三大组件，具体如下。

1. 修改配置类

将文件 3-5 中 WebConfigure 类原有的方法进行注释 ,将自定义的 Servlet、Filter 和 Listener

包装到对应的 RegistrationBean 中，并使用@Bean 注解将 ServletRegistrationBean、FilterRegistrationBean 和 ServletListenerRegistrationBean 注册到 Spring 容器中。修改后代码如文件 3-6 所示。

文件 3-6 WebConfigure.java

```
1  import com.itheima.chapter03.web.FirstServlet;
2  import com.itheima.chapter03.web.MyFilter;
3  import com.itheima.chapter03.web.MyListener;
4  import com.itheima.chapter03.web.SecondServlet;
5  import org.springframework.boot.web.servlet.FilterRegistrationBean;
6  import org.springframework.boot.web.servlet.
7         ServletListenerRegistrationBean;
8  import org.springframework.boot.web.servlet.ServletRegistrationBean;
9  import org.springframework.context.annotation.Bean;
10 import org.springframework.context.annotation.Configuration;
11 import java.util.Arrays;
12 @Configuration
13 public class WebConfigure {
14     @Bean
15     public ServletRegistrationBean firstServlet(){
16         return  new ServletRegistrationBean(new FirstServlet(),"/first");
17     }
18     @Bean
19     public ServletRegistrationBean secondServlet(){
20         return  new ServletRegistrationBean(new SecondServlet(),"/second");
21     }
22     @Bean
23     public FilterRegistrationBean myFilter(){
24         FilterRegistrationBean filterRegistrationBean =
25             new FilterRegistrationBean(new MyFilter());
26         filterRegistrationBean.setUrlPatterns(Arrays.asList("/first"));
27         return  filterRegistrationBean;
28     }
29     @Bean
30     public ServletListenerRegistrationBean myListener(){
31         return  new ServletListenerRegistrationBean(new MyListener());
32     }
33 }
```

在上述代码中，第 16 行和第 20 行代码分别使用 ServletRegistrationBean 对 FirstServlet 和 SecondServlet 进行包装，指定访问这两个 Servlet 的路径为 first 和 second。第 24～26 行代码使用 FilterRegistrationBean 对 MyFilter 进行包装，并指定拦截的路径为 “ /first ”。

2. 测试程序效果

运行项目的启动类，控制台输出如图 3-3 所示。

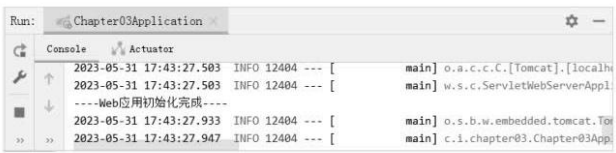

图3-3 控制台输出（3）

从图 3-3 可以看出，控制台输出 Web 应用初始化完成等信息，说明项目启动时，Web 服务器中成功注册了 MyListener ,在监听到 Web 应用初始化时 ,执行了对应的方法进行输出。

在浏览器中访问 http://localhost:8080/first，此时控制台输出如图 3-4 所示。

从图 3-4 可以看出，在浏览器中访问 FirstServlet 时，项目中的拦截器拦截到对应的请求，在放行后执行了 FirstServlet 的 doGet()方法。

在浏览器中访问 http://localhost:8080/second，此时控制台输出如图 3-5 所示。

图3-4　控制台输出（4）

图3-5　控制台输出（5）

从图 3-5 可以看出，在浏览器中访问 SecondServlet 时，项目中的拦截器没有对该资源的访问进行拦截，说明注册的 Listener 拦截路径设置成功。

3.1.3　使用注解扫描注册 Java Web 三大组件

Spring Boot 无法自动识别到@WebServlet、@WebFilter、@WebListener 标注的类，但其内部可使用嵌入式容器 ,可以使用@ServletComponentScan 扫描标注@WebServlet、@WebFilter 和@WebListener 的类，并将扫描到的类自动注册到 Spring 容器。下面通过案例演示使用注解扫描注册 Java Web 三大组件，具体如下。

1. 使用注解声明组件

分别使用@WebServlet、@WebFilter、@WebListener 标注 FirstServlet 类、MyFilter 类和 MyListener 类，具体如文件 3-7～文件 3-9 所示。

文件 3-7　FirstServlet.java

```
1  import javax.servlet.annotation.WebServlet;
2  import javax.servlet.http.HttpServlet;
3  import javax.servlet.http.HttpServletRequest;
4  import javax.servlet.http.HttpServletResponse;
5  import java.io.IOException;
6  @WebServlet("/first")
7  public class FirstServlet extends HttpServlet {
8      @Override
9      public void doGet(HttpServletRequest req, HttpServletResponse resp)
10         throws  IOException {
11         System.out.println("hello FirstServlet");
12         resp.getWriter().write("hello FirstServlet");
13     }
14 }
```

在上述代码中，第 6 行代码使用@WebServlet 注解声明自定义的 Servlet，并指定对应的映射路径为“ /first ”。

文件 3-8　MyFilter.java

```
1  import javax.servlet.*;
```

```
2  import javax.servlet.annotation.WebFilter;
3  import java.io.IOException;
4  @WebFilter("/first")
5  public class MyFilter  implements Filter {
6      @Override
7      public void doFilter(ServletRequest requ, ServletResponse resp,
8          FilterChain filterChain) throws IOException, ServletException {
9          System.out.println("处理请求前的处理");
10         filterChain.doFilter(requ,resp);
11         System.out.println("处理请求后的处理");
12     }
13 }
```

在上述代码中，第 4 行代码使用@WebFilter 注解声明自定义的 Filter，并指定拦截的路径为“ /first ”。

文件 3-9　MyListener.java

```
1  import javax.servlet.ServletContextEvent;
2  import javax.servlet.ServletContextListener;
3  import javax.servlet.annotation.WebListener;
4  @WebListener
5  public class MyListener implements ServletContextListener {
6      @Override
7      public void contextInitialized(ServletContextEvent sce) {
8          System.out.println("----Web 应用初始化完成----");
9      }
10     @Override
11     public void contextDestroyed(ServletContextEvent sce) {
12         System.out.println("----Web 应用销毁之前----");
13     }
14 }
```

在上述代码中，第 4 行代码使用@WebListener 注解声明自定义的 Listener。

2. 添加@ServletComponentScan 注解

在项目的启动类上添加@ServletComponentScan 注解，具体如文件 3-10 所示。

文件 3-10　Chapter03Application.java

```
1  import org.springframework.boot.SpringApplication;
2  import org.springframework.boot.autoconfigure.SpringBootApplication;
3  import org.springframework.boot.web.servlet.ServletComponentScan;
4  @ServletComponentScan
5  @SpringBootApplication
6  public class Chapter03Application {
7      public static void main(String[] args) {
8          SpringApplication.run(Chapter03Application.class, args);
9      }
10 }
```

在上述代码中，在 Chapter03Application 类上使用@ServletComponentScan 注解进行标注，在项目启动时扫描当前类所在包以及子包，将标记@WebServlet、@WebFilter 和 @WebListener 注解的组件类注册到 Spring 容器中。

3. 测试程序效果

使用扫描组件的方式注释原生组件不需要其他配置类，因此，先注释掉文件 3-6 中的配置类 WebConfigure，然后运行项目的启动类，控制台输出如图 3-6 所示。

图3-6　控制台输出（6）

从图 3-6 可以看出，控制台输出 Web 应用初始化完成等信息，说明项目启动时，Web 服务器中成功注册了 MyListener。

在浏览器中访问 http://localhost:8080/first，此时控制台输出如图 3-7 所示。

图3-7　控制台输出（7）

从图 3-7 可以看出，浏览器中访问 FirstServlet 时，项目中的拦截器拦截到对应的请求，在放行后执行了 FirstServlet 的 doGet()方法，说明项目中成功注册了 FirstServlet 和 MyFilter。

3.2　Spring Boot 管理 Spring MVC

Spring Boot 真正的核心功能是自动配置和快速整合，通常 Spring Boot 应用的前端 MVC 框架依然使用 Spring MVC。Spring Boot 提供的 spring-boot-starter-web 启动器嵌入了 Spring MVC 的依赖，并为 Spring MVC 提供了大量自动配置，可以适用于大多数 Web 开发场景。除了使用自动配置所提供的功能，开发者也可以通过自定义配置类定制 Spring MVC 的配置。下面分别对 Spring MVC 自动配置的特性和自定义 Spring MVC 配置进行讲解。

3.2.1　Spring MVC 自动配置的特性

Spring Boot 为 Spring MVC 提供了自动配置，并在 Spring MVC 默认功能的基础上添加了以下特性。

（1）引入了视图解析器 ContentNegotiatingViewResolver 和 BeanNameViewResolver。

（2）为包括 WebJars 在内的静态资源提供支持。

（3）自动注册 Converter、GenericConverter 和 Formatter。

（4）支持使用 HttpMessageConverters 消息转换器。

（5）自动注册 MessageCodesResolver。

（6）支持静态项目首页 index.html。

（7）支持定制应用图标 favicon.ico。

（8）自动初始化 Web 数据绑定器 ConfigurableWebBindingInitializer。

在 Spring Boot 应用中使用 Spring MVC 只需在项目的 pom.xml 中引入 spring-boot-starter-web 启动器，即可直接使用 Spring MVC 进行便捷的 Web 开发，例如不用自行配置视图解析器、消息转换器等。

3.2.2 自定义 Spring MVC 配置

在 Spring Boot 应用中使用 Spring MVC 时，如果希望在为 Spring MVC 自动配置提供相关特性的同时，再增加一些自定义的 Spring MVC 配置，例如添加拦截器、视图控制器等，可以通过自定义 WebMvcConfigurer 类型的配置类来实现。下面分别对自定义 Spring MVC 配置中的配置静态资源映射、配置视图控制器、配置拦截器进行讲解。

1. 配置静态资源映射

通常 Web 应用中会需要使用静态资源，例如，JavaScript 文件、CSS 文件和 HTML 文件等。单独使用 Spring MVC 时，导入静态资源文件后，需要配置静态资源的映射，否则无法正常访问。Spring Boot 中提供了默认的静态资源映射，当访问项目中任意的静态资源时，Spring Boot 会默认从以下路径中进行查找。

（1）classpath:/META-INF/resources/。

（2）classpath:/resources/。

（3）classpath:/static/。

（4）classpath:/public/。

上述路径对应的文件夹又被称为静态资源文件夹，查找时的优先级从（1）至（4）依次递减，Spring Boot 会先查找优先级高的文件夹，再查找优先级低的文件夹，直到找到指定的静态资源。

下面在 chapter03 项目中的 src/main/resources/static 和 src/main/resources 目录下分别创建 main.html 文件和 index.html 文件，并在项目启动后分别在浏览器中对这两个静态资源进行访问，访问结果如图 3-8 和图 3-9 所示。

图3-8 访问main.html

图3-9 访问index.html

从图 3-8 和图 3-9 可以看出，项目启动后可以直接访问到静态资源文件夹下的资源，而访问不到其他文件夹下的资源。

如果想在项目中访问非默认静态资源文件夹下的资源，可以自定义静态资源的映射。自定义静态资源的映射可以通过配置类和配置文件这两种方式实现，具体如下。

（1）通过配置类实现静态资源映射

通过配置类实现静态资源映射时，配置类需要实现 WebMvcConfigurer 接口，在重写 WebMvcConfigurer 接口的 addResourceHandlers()方法中指定资源访问路径和资源之间的映射关系。addResourceHandlers()方法的形参为 ResourceHandlerRegistry 对象，该对象用于保存静态资源的资源处理器的注册信息，使用该对象的 addResourceHandlers()方法可以添加资源的访问路径，addResourceLocations()方法可以添加资源路径映射的真实路径。

（2）通过配置文件实现静态资源映射

Spring Boot 在 Spring MVC 的自动配置中提供了对应的属性可以配置静态资源访问路径和资源的映射，示例如下。

```
spring:
  mvc:
    static-path-pattern: /backend/**
  web:
    resources:
      static-locations:
file:E:\idea\SpringBoot\chapter03\src\main\resources\backend
```

在上述代码中，static-path-pattern 用于指定静态资源的访问路径，也就是访问静态资源的规则，这里表示只有静态资源的访问路径为/backend/**时才会处理请求，其中**表示匹配所有；static-locations 用于指定静态资源存放的目录，其中 file 表示映射的为本地文件。配置上述内容后，访问路径为/backend/**的静态资源时，会在本地的 E:\idea\SpringBoot\chapter03\src\main\resources\backend 目录下查找。

下面以通过配置类实现静态资源映射为例，演示配置静态资源映射，具体如下。

（1）创建静态资源。在 chapter03 项目中的 src/main/resources 目录下创建文件夹 backend，并在文件夹中创建 HTML 文件 index.html 和 login.html，在两个页面的<body>标签中书写任意文字，以便于后续对两个页面的展示进行区分，静态资源的目录结构如图 3-10 所示。

（2）配置静态资源映射。在项目 chapter03 的 com.itheima.chapter03.config 包下创建配置类 WebMvcConfig，该配置类实现了 WebMvcConfigurer 接口，并重写了该接口的方法以实现自定义 Spring MVC 的配置，具体如文件 3-11 所示。

文件 3-11　WebMvcConfig.java

```
1  import org.springframework.context.annotation.Configuration;
2  import org.springframework.web.servlet.config.annotation.
3         ResourceHandlerRegistry;
4  import org.springframework.web.servlet.config.annotation.
5         WebMvcConfigurer;
6  @Configuration
7  public class WebMvcConfig implements WebMvcConfigurer {
8     @Override
9     public void addResourceHandlers(ResourceHandlerRegistry registry) {
```

```
10        registry.addResourceHandler("/backend/**")
11                .addResourceLocations("classpath:/backend/");
12    }
13}
```

在上述代码中，第 6 行代码使用@Configuration 注解标注当前类为配置类；第 9～12 行代码重写了 WebMvcConfigurer 接口的 addResourceHandlers()方法，其中，第 10～11 行代码设置了访问以“ /backend ”开头的所有路径时，映射到 classpath 的 backend 文件夹下。

（3）测试程序效果。启动项目，在浏览器中访问 backend 文件夹下的 index.html，效果如图 3-11 所示。

图3-11　访问index.html

从图 3-11 可以看出，可以正常访问 index.html 页面，说明可成功访问默认静态资源文件夹之外的静态资源，自定义静态资源映射配置成功。

2．配置视图控制器

使用 Spring MVC 默认的配置进行开发时，如果仅需要实现无业务逻辑的页面跳转，也需要创建 Controller 类，然后定义方法跳转到页面，操作比较复杂。对此，可以在视图控制器中添加自定义的映射，直接将请求映射为视图。

下面通过案例演示在视图控制器中配置请求和视图的映射，具体如下。

（1）配置视图控制器映射信息。在文件 3-11 中重写 WebMvcConfigurer 接口的 addViewControllers()方法，在该方法中添加访问路径和视图的映射，具体如下。

```
@Override
public void addViewControllers(ViewControllerRegistry registry) {
   registry.addViewController("/backend/toLoginPage")
.setViewName("/backend/login.html");
   registry.addViewController("/backend")
.setViewName("/backend/index.html");}
```

在上述代码中，为访问路径为“ /backend/toLoginPage ”的控制器设置视图为“ /backend/login.html ”，以及为访问路径为“ /backend ”的控制器设置视图为“ /backend/index.html ”。即访问“ http://localhost:8080/backend/toLoginPage ”时，会自动跳转到“ http://localhost:8080/backend/login.html ”，访问“ http://localhost:8080/backend ”时，会自动跳转到“ http://localhost:8080/backend/index.html ”。

（2）测试程序效果。启动项目，在浏览器中访问 http://localhost:8080/toLoginPage，效果如图 3-12 所示。

图3-12　访问toLoginPage

从图 3-12 可以看出，访问 toLoginPage 时直接跳转到 login.html 页面，说明自定义视图控制器配置成功。这种方式适用于需要页面跳转而没有具体的业务逻辑请求处理的情况，

相较于传统的请求处理方法而言，这种方法更加简洁、直观和方便。

3. 配置拦截器

拦截器可以根据请求的 URL 对请求进行拦截，主要应用于登录校验、权限验证、乱码解决、性能监控和异常处理等方面。在 Spring Boot 项目中配置拦截器也非常简单，只需要定义拦截器和注册拦截器即可，下面通过案例演示在 Spring Boot 项目中配置拦截器，具体如下。

（1）定义拦截器。在项目 chapter03 下创建 com.itheima.chapter03.interceptor 包，在该包下创建拦截器类，该类实现了 HandlerInterceptor 接口，并重写了接口的 preHandle()方法，具体如文件 3-12 所示。

文件 3-12　LoginInterceptor.java

```
1  import org.springframework.web.servlet.HandlerInterceptor;
2  import javax.servlet.http.HttpServletRequest;
3  import javax.servlet.http.HttpServletResponse;
4  public class LoginInterceptor implements HandlerInterceptor {
5      @Override
6      public boolean preHandle(HttpServletRequest request,
7          HttpServletResponse response, Object handler) throws Exception {
8          Object loginUser = request.getSession().getAttribute("loginUser");
9          if (loginUser == null) {
10             //未登录，返回登录页
11             request.getRequestDispatcher("backend/login.html")
12                    .forward(request, response);
13             return false;
14         } else {
15             //放行
16             return true;
17         }
18     }
19 }
```

在上述代码中，通过实现 HandlerInterceptor 接口定义了拦截器类。其中，第 6～18 行代码会在处理请求之前执行，代码中判断 Session 中是否存在用户信息，如果不存在，则判定用户未登录，将请求转发到登录页面；如果存在，则放行。

（2）注册拦截器。在文件 3-11 中重写 WebMvcConfigurer 接口的 addInterceptors()方法，在该方法中添加拦截器，具体如下。

```
@Override
public void addInterceptors(InterceptorRegistry registry) {
//拦截所有请求，包括静态资源文件
    registry.addInterceptor(new oginInterceptor()).addPathPatterns("/**")
        .excludePathPatterns("/backend/login.html");
}
```

在上述代码中，使用 addPathPatterns()方法和 excludePathPatterns()方法指定拦截器拦截的规则。其中，addPathPatterns()方法用于指定拦截路径，例如拦截路径为“/**”，表示拦截所有请求，包括对静态资源的请求；excludePathPatterns()方法用于排除拦截路径，即指定不需要被拦截器拦截的请求。

（3）测试程序效果。启动项目，在浏览器中访问 http://localhost:8080/backend/login.html，

效果如图 3-13 所示。

图3-13 访问login.html页面

从图 3-13 可以看出，可以正常访问到 login.html 页面，说明拦截器中成功放行了该访问路径。

在浏览器中访问 http://localhost:8080/backend/index.html，效果如图 3-14 所示。

图3-14 访问index.html页面

从图 3-14 可以看出，页面跳转到了 login.html 页面，说明访问 index.html 页面时，检测到当前 Session 中没有登录的用户信息，则对该路径进行拦截，并根据代码中的业务，将请求转发到 login.html 页面，说明所配置的自定义的拦截器已经生效。

需要说明的是，Spring Boot 在整合 Spring MVC 过程中提供了许多默认自动化配置和特性，开发者可以通过 Spring Boot 提供的 WebMvcConfigurer 接口对 MVC 功能进行定制和扩展。如果开发者不想使用 Spring Boot 整合 MVC 时提供的一些默认配置，而是想要完全自定义管理，那么可以编写一个@Configuration 注解配置类，同时添加@EnableWebMvc 注解关闭 Spring Boot 提供的所有关于 MVC 功能的默认配置。

3.3 文件上传

在项目开发过程中，文件的上传和下载是比较常见的开发需求。Spring Boot 为 Spring MVC 的文件上传同样提供了自动配置，Spring Boot 推荐使用基于 Servlet 3 的文件上传机制，这样可直接利用 Web 服务器内部的文件上传支持，而无须引入第三方 JAR 包。

Spring Boot 的文件上传自动配置主要由 MultipartAutoConfiguration 类和 Multipart Properties 类组成，其中 MultipartProperties 负责加载以“ spring.servlet.multipart ”开头的配置属性，而 MultipartAutoConfiguration 则根据 MultipartProperties 读取的配置属性来初始化 StandardServlet MultipartResolver 解析器对象。

文件上传功能在很多系统中都占据非常重要的位置，软件开发人员在实现文件上传时，除了需要确保程序代码的稳健性和高维护性外，还需要从社会责任的角度审视程序的需求，秉承合法、合规的职业理念进行数据读写，避免程序在保存用户上传的文件时造成不良的社会影响。例如，应该关注保护用户的隐私，避免用户隐私泄露，以及杜绝上传非法文件。

下面通过案例演示在 Spring Boot 项目中进行文件上传，该案例演示的是图片文件的上传，图片上传后将图片显示在上传页面，具体如下。

1. 设置上传配置

在 chapter03 项目的 application.yml 文件中对静态资源的映射和文件上传进行配置，具

体如下。

```
spring:
  mvc:
    static-path-pattern: /backend/**
  web:
    resources:
      static-locations:
file:E:\idea\SpringBoot\chapter03\src\main\resources\backend
  servlet:
    multipart:
      max-file-size: 10MB
      max-request-size: 50MB
```

上述配置属性在自动配置中也提供了默认配置，其中，spring.servlet.multipart. max-file-size 用于设置单个上传文件的大小限制，默认值为 1MB；spring.servlet.multipart.max- request-size 用于设置所有上传文件的大小限制，默认值为 10MB。如果上传文件的大小超出默认值 10MB，会出现“ FileUploadBase$FileSizeLimitExceededException: The field fileUpload exceeds its maximum permitted size of 1048576 bytes ”异常信息。

2. 创建文件上传页面

在项目中的 src/main/resources 目录下创建 HTML 页面用于操作文件上传，具体如文件 3-13 所示。

文件 3-13　fileupload.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
  <script src="js/jquery.min.js"></script>
</head>
<body>
<form id="myform" method="post" enctype="multipart/form-data">
<table style="border: thin solid">
  <tr>
    <td style="width: 200px;height: 200px" >
      <img id="myimg" src="" >
    </td>
    <td>
      <input type="file" name="files"  multiple="multiple">
    </td>
  </tr>
  <tr>
    <td colspan="2" style="width: 200px;height: 50px;
        text-align:center ; border: thin solid">
      <input type="button"  value="提交" onclick="uploadAction()"/>
    </td>
  </tr>
</table>
</form>
<script>
```

```
28  function uploadAction()
29  {
30    $.ajax({
31      url:"../fileUpload",
32      type: "POST",
33      cache : false,
34      processData: false,
35      contentType: false,
36      dataType:"json",
37      data: new FormData($('#myform')[0]),
38      success: function(data){
39        if(data.flag==true){
40          $("#myimg").attr("style","width: 200px;height: 200px");
41          $("#myimg").attr("src",data.message);
42        }else{
43          $("#myimg").removeAttr("src");
44          $("#myimg").attr("alt",data.message);
45        }
46      }
47    });
48  }
49 </script>
50 </body>
51 </html>
```

在上述代码中，第 9～26 行代码定义了一个表单，其中，<form>表单中必须设置 enctype 属性的值为“ multipart/form-data ”，并且请求方式为 post，第 16 行和第 22 行代码分别定义了文件选择控件和按钮控件，并为按钮控件绑定了鼠标单击事件。第 28～48 行代码定义了 uploadAction()方法，该方法在单击按钮时触发，触发时异步提交表单，如果提交成功，则将表单中上传的图标显示在页面中。

3. 创建文件上传控制器类

在项目 chapter03 的 com.itheima.chapter03.controller 包下创建控制器类 FileController，在该类中处理文件上传的请求，将上传的文件存放在指定路径下，并返回上传结果，具体如文件 3-14 所示。

文件 3-14 FileController.java

```
1  import com.itheima.chapter03.entity.Result;
2  import org.springframework.web.bind.annotation.RequestMapping;
3  import org.springframework.web.bind.annotation.RestController;
4  import org.springframework.web.multipart.MultipartFile;
5  import javax.servlet.http.HttpServletRequest;
6  import java.io.File;
7  import java.util.UUID;
8  @RestController
9  public class FileController {
10    @RequestMapping("/fileUpload")
11    public Result fileUpload(MultipartFile[] files ,
12        HttpServletRequest request) {
13        for (MultipartFile file : files) {
14            String fileName = file.getOriginalFilename();
```

```
15          String subfix =fileName.substring(fileName.lastIndexOf("."));
16          if(!".jpg".equalsIgnoreCase(subfix)){
17             return new Result(false, "上传失败，只能上传 jpg 格式的图片！");
18          }
19          // 重新生成文件名（根据具体情况生成对应文件名）
20          fileName = UUID.randomUUID() + subfix;
21          //设置上传的文件所存放的路径
22          String dirPath="src/main/resources/backend/upload";
23          File filePath = new File(dirPath);
24          if (!filePath.exists()) {
25             filePath.mkdirs();
26          }
27          try {
28             file.transferTo(new File(filePath.getCanonicalFile() +
29               "/"+fileName));
30             return new Result(true, "/backend/upload/"+fileName);
31          } catch (Exception e) {
32             e.printStackTrace();
33             // 上传失败，返回失败信息
34             return new Result(false, "上传失败！");
35          }
36       }
37       return new Result(false, "上传失败！");
38    }
39 }
```

在上述代码中，第 11 行代码使用 MultipartFile[]接收上传的文件；第 15 行代码获取上传文件的后缀名；第 16～18 行代码判断上传文件的后缀名是不是.jpg，不是则返回上传失败的提示；第 20 行代码获取随机的 UUID 和文件后缀名组合为新的文件名称；第 22 行代码指定文件上传后的存放路径；第 28～30 行代码用于保存上传的文件，并返回该文件的访问路径。

4. 程序效果测试

为了便于测试，先将 chapter03 项目中拦截器相关配置进行注释，启动项目，在浏览器访问 fileupload.html，如图 3-15 所示。

图3-15　访问fileupload.html页面

单击“选择文件”按钮，选择一个后缀名不是.jpg 的文件，然后单击“提交”按钮，效果如图 3-16 所示。

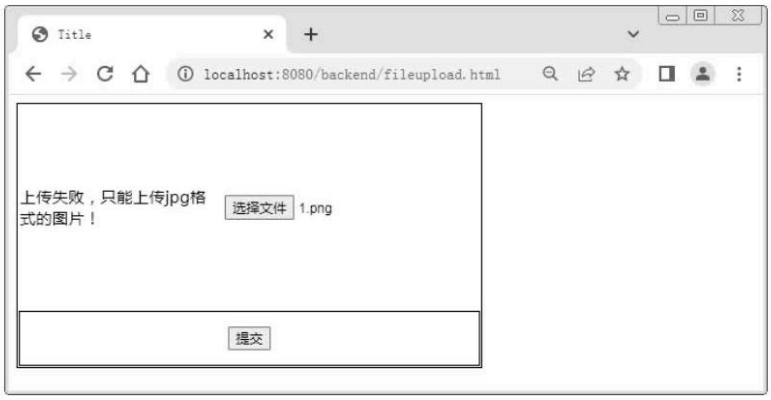

图3-16　上传后缀名不是.jpg的文件

从图 3-16 可以看出，页面给出上传失败的提示。

再次单击“选择文件”按钮，选择后缀名是.jpg 的文件，然后单击“提交”按钮，效果如图 3-17 所示。

图3-17　上传后缀名是.jpg的文件

从图 3-17 可以看出，页面中展示了上传的图片，此时项目 src/main/resources 目录下多了一个 upload 文件夹，文件夹下存放了刚才上传的文件，说明文件上传成功，并且在浏览器中获取到指定静态资源文件夹中的图片。

3.4　异常处理

在日常的 Web 开发中，项目中难免会出现各种异常，为了使客户端能接收较为友好的提示，通常开发者会对异常进行统一处理。为了便于开发者处理异常，Spring Boot 通过自动配置提供了一套默认的异常处理机制，一旦程序中出现了异常，Spring Boot 会根据该机制进行默认的异常处理。除了默认的异常处理，Spring Boot 也支持自定义异常处理。下面对 Spring Boot 中的异常处理进行讲解。

3.4.1　Spring Boot 异常处理自动配置原理

Spring Boot 通过配置类 ErrorMvcAutoConfiguration 为异常处理提供了自动配置，该配置

类向容器中注入了以下 4 个组件。

- ErrorPageCustomizer：错误页面响应规则类，该组件会在系统发生异常后，默认将请求转发到“/error”。
- BasicErrorController：错误控制器，处理“/error”请求。
- DefaultErrorViewResolver：默认的错误视图解析器，将异常信息解析到相应的错误视图。
- DefaultErrorAttributes：错误属性处理类，用于页面上共享异常信息。

Spring Boot 异常处理自动配置主要基于上述 4 个组件，下面对这 4 个组件进行讲解，具体如下。

1. ErrorPageCustomizer

Spring Boot 项目中配置了 Spring MVC 的启动器后，项目启动时会加载异常处理的自动配置类 ErrorMvcAutoConfiguration，ErrorMvcAutoConfiguration 会向容器中注入一个名为 ErrorPageCustomizer 的组件，该组件用于定制错误页面的响应规则。

ErrorPageCustomizer 类中提供了 registerErrorPages()方法用于注册错误页面的响应规则。当系统发生异常后，ErrorPageCustomizer 组件会自动生效，并将请求转发到 Spring Boot 的异常处理地址，默认为“ /error ”。

2. BasicErrorController

ErrorMvcAutoConfiguration 还向容器中注入了错误控制器组件 BasicErrorController，BasicErrorController 中会对异常处理路径进行统一映射处理。BasicErrorController 提供了 errorHtml()和 error()方法。其中，errorHtml()方法用于处理请求的 MediaType 为 text/html 的请求，并使用错误视图解析器生成包含错误页面地址和页面内容的 ModelAndView 对象；error()方法用于处理其他的请求，并返回 JSON 格式的数据展示错误信息。

3. DefaultErrorViewResolver

ErrorMvcAutoConfiguration 加载的同时也会向容器中注入 DefaultErrorViewResolver 类。当 BasicErrorController 使用 errorHtml()方法处理异常请求时，Spring Boot 会获取容器中所有的 ErrorViewResolver（错误视图解析器）对象，并分别调用 resolveErrorView()方法对异常信息进行解析，其中默认错误信息解析器为 DefaultErrorViewResolver。

DefaultErrorViewResolver 解析异常信息的步骤如下。

根据错误状态码（如 404、500、400 等）生成对应的错误视图，错误视图的名称格式为“ error/status ”，其中 status 为错误状态码，例如 error/404、error/500、error/400。

尝试使用模板引擎解析错误视图。从 classpath 类路径的 templates/error 目录下查找 status.html 文件，其中 status 为错误状态码，例如 404.html、500.html、400.html。若模板引擎能够解析到对应的视图，则将视图和数据封装成 ModelAndView 返回，并结束整个解析流程。

如果模板引擎不能正确解析到对应的视图，则依次从各个静态资源文件夹的 error 文件夹中查找 status.html 文件，若在静态文件夹中找到了该错误页面，则将视图和数据封装成 ModelAndView 返回，并结束整个解析流程。

如果上述流程都没有查找到对应的视图，则使用 Spring Boot 默认的错误页面作为响应的视图。

4. DefaultErrorAttributes

ErrorMvcAutoConfiguration 还向容器中注入了 DefaultErrorAttributes 类，DefaultErrorAttributes 是 Spring Boot 的默认错误属性处理类，它可以从请求中获取异常或错误信息，并将其封装为一个 Map 对象返回。

在 BasicErrorController 处理异常时，会调用 DefaultErrorAttributes 的 getErrorAttributes() 方法获取错误或异常信息，并封装到 Model 中，返回到页面或 JSON 数据中。该 Model 中主要包含以下属性。

- timestamp：时间戳。
- status：错误的状态码。
- error：错误的原因描述。
- exception：导致请求处理失败的异常类名。
- message：错误/异常消息。
- trace：错误/异常栈信息。
- path：错误/异常抛出时所请求的 URL 路径。

3.4.2 Spring Boot 自定义异常处理

Spring Boot 默认的异常处理其实就是对 Spring MVC 异常处理的自动配置。如果想要自定义异常处理，可以在 Spring Boot 提供的自动配置的基础上，通过修改配置信息以修改 Spring Boot 默认的错误处理行为，或者使用 @ResponseStatus、@ExceptionHandler、@ControllerAdvice 等注解基于 Spring MVC 异常处理机制进行异常处理。

读者如果对 Spring MVC 异常处理机制不熟悉，可以参考黑马程序员编著的《Java EE 企业级应用开发教程（Spring+Spring MVC+MyBatis）（第 2 版）》进行学习，在此就不再对 Spring MVC 的相关内容进行进一步讲解。

安全稳定是应用程序的关键，开发人员在开发应用程序时，应在确保程序可用的基础上，时刻紧绷安全稳定的“思想弦”，保障代码安全、程序稳定，并为有可能出现的异常问题制定合理的解决方案。下面通过案例演示使用@ExceptionHandler 和@ControllerAdvice 完成 Spring Boot 全局异常处理，具体如下。

1. 创建自定义异常类

在项目的 com.itheima.chapter03.exception 包下创建自定义异常类，具体如文件 3-15 所示。

文件 3-15 MyException.java

```
1  public class MyException extends  RuntimeException{
2      //异常信息
3      private String message;
4      public MyException(String message) {
5          super(message);
6          this.message = message;
7      }
8      @Override
9      public String getMessage() {
10         return message;
11     }
12     public void setMessage(String message) {
```

```
13        this.message = message;
14    }
15 }
```

2. 创建异常处理类

在 com.itheima.chapter03.handler 包下创建异常处理器 MyExceptionHandler，在该类中定义了两个方法，分别处理控制器执行时抛出的 Exception 异常和自定义异常，具体如文件 3-16 所示。

文件 3-16　MyExceptionHandler.java

```
1  import com.itheima.chapter03.exception.MyException;
2  import org.springframework.http.HttpStatus;
3  import org.springframework.http.ResponseEntity;
4  import org.springframework.web.bind.annotation.ControllerAdvice;
5  import org.springframework.web.bind.annotation.ExceptionHandler;
6  @ControllerAdvice
7  public class MyExceptionHandler {
8     @ExceptionHandler(Exception.class)
9     public ResponseEntity<String> exceptionHandler(Exception e) {
10       return new ResponseEntity<>(e.getMessage(),
11             HttpStatus.INTERNAL_SERVER_ERROR);
12    }
13    @ExceptionHandler(MyException.class)
14    public ResponseEntity<String> exceptionHandler(MyException e) {
15       return new ResponseEntity<>(e.getMessage(),
16             HttpStatus.INTERNAL_SERVER_ERROR);
17    }
18 }
```

在上述代码中，第 6 行代码对控制器进行增强，可以为控制器添加统一的处理。第 8 行和第 13 行代码使用@ExceptionHandler 注解声明当前方法处理的异常类型 ,当控制器抛出对应异常之后，当前方法会对这些异常进行捕获。exceptionHandler()方法中将异常的信息添加在 ResponseEntity 中返回。

3. 创建控制器类

在 com.itheima.chapter03.controller 包下创建控制器类 DishController，在该类中定义 delDish()方法处理请求，具体如文件 3-17 所示。

文件 3-17　DishController.java

```
1  import com.itheima.chapter03.exception.MyException;
2  import org.springframework.http.HttpStatus;
3  import org.springframework.http.ResponseEntity;
4  import org.springframework.web.bind.annotation.PathVariable;
5  import org.springframework.web.bind.annotation.RequestMapping;
6  import org.springframework.web.bind.annotation.RestController;
7  @RestController
8  @RequestMapping("/dish")
9  public class DishController {
10    @RequestMapping("/del/{i}")
11    public ResponseEntity<String> delDish(@PathVariable String i)  {
12       Integer id=Integer.valueOf(i);
13       if(id<0){
```

```
14            throw  new MyException("参数错误！");
15        }
16        return new ResponseEntity<>("del success", HttpStatus.OK);
17    }
18 }
```

在上述代码中，第 11 行代码接收请求参数并封装在变量 i 中；第 12 行代码将 i 转换为 Integer 类型；第 13～15 行代码判断 id 的值，如果小于 0，则抛出自定义异常，如果程序没有抛出异常，则执行第 16 行代码，表示请求成功。

4. 测试全局异常处理

启动程序，在浏览器中访问 http://localhost:8080/dish/del/1，请求结果如图 3-18 所示。

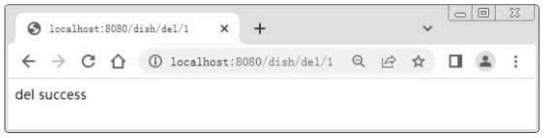

图3-18　请求结果（1）

从图 3-18 可以看出，控制器成功对请求进行处理并响应。

将请求删除的参数修改为-1，在浏览器中访问 http://localhost:8080/dish/del/-1，请求结果如图 3-19 所示。

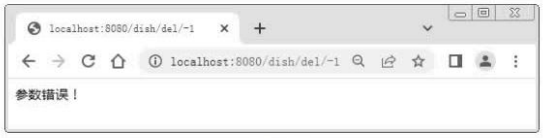

图3-19　请求结果（2）

从图 3-19 可以看出，当控制器判断参数为负数时，抛出了自定义异常，异常处理类中对该自定义异常进行捕获后，将异常信息响应回浏览器。

将请求删除的参数修改为字母，在浏览器中访问 http://localhost:8080/dish/del/p，请求结果如图 3-20 所示。

图3-20　请求结果（3）

从图 3-20 可以看出，当请求参数无法转换为数字时，抛出异常，异常处理类对该自定义异常进行捕获后，将异常信息响应回浏览器。

3.5　本章小结

本章主要对 Spring Boot 的 Web 应用支持进行了讲解。首先讲解了 Spring Boot 注册 Java Web 三大组件；然后讲解了 Spring Boot 管理 Spring MVC；接着讲解了文件上传；最后讲解了异常处理。通过本章的学习，希望大家可以对 Spring Boot 的 Web 应用支持有所了解，为

后续更深入学习 Spring Boot 做好铺垫。

3.6　本章习题

一、填空题

1．使用 Spring Bean 注册 Filter 时，Filter 的映射地址默认为_____。

2．Spring Boot 中可以使用_____扫描到标注@WebServlet、@WebFilter 和@WebListener 的类。

3．InterceptorRegistry 类的_____方法用于指定拦截路径。

4．Spring Boot 整合 Spring MVC 实现文件上传时，<form>表单请求方式需要为_____请求。

5．_____注解可以声明当前方法处理的异常类型。

二、判断题

1．使用 Spring Bean 注册自定义 Servlet 时，Servlet 对应的映射地址为“ Bean 名称+/ ”。(　　)

2．Spring Boot 默认情况下无法自动识别到@WebServlet、@WebFilter、@WebListener 标注的类。(　　)

3．Spring Boot 项目中非静态资源文件夹中的静态资源不能被外部直接访问。(　　)

4．Spring Boot 项目中，BasicErrorController 会对异常处理路径进行统一映射处理。(　　)

5．在 Spring Boot 应用中引入 spring-boot-starter-web 启动器后，即使不进行任何配置，也会自行配置视图解析器。(　　)

三、选择题

1．下列选项中，关于使用 Spring Bean 注册 Java Web 三大组件描述错误的是(　　)。

A．在 Spring Boot 项目中，会自动将 Spring 容器中的 Servlet、Filter、Listener 实例注册为 Web 服务器中对应的组件。

B．使用 Spring Bean 注册自定义的 Servlet 时，Servlet 对应的映射地址为“ Bean 名称+/ ”。

C．Filter 的映射地址默认为“ / ”。

D．使用 Spring Bean 注册自定义 Servlet 时，需要两个及以上的 Servlet。

2．下列选项中，使用 Spring Bean 注册 Filter 时，Filter 默认的映射地址是(　　)。

A．/Bean 名称　　B．/*

C．./　　D．/**

3．下列选项中，关于 Spring Boot 整合 Spring MVC 实现文件上传描述错误的是(　　)。

A．配置信息中 spring.servlet.multipart.max-file-size 用于设置单个上传文件的大小限制。

B．配置信息中 spring.servlet.multipart.max-request-size 用于设置所有上传文件的大小限制。

C．spring.servlet.multipart.max-request-size 默认为 10MB。

D．上传文件的大小超出所设置的上传文件的最大限制，则上传失败，但不会抛出异常信息。

4．下列选项中，关于 WebMvcConfigurer 接口描述错误的是(　　)。

A．addResourceHandlers()方法可以添加资源路径映射的真实路径。

B．addViewControllers()方法中可以添加访问路径和视图的映射。

C．addInterceptors()方法可以添加拦截器。

D．通过配置类实现静态资源映射时，配置类需要实现 WebMvcConfigurer 接口。

5．下列选项中，Spring Boot 默认的错误信息解析器是（　　）。

A．ErrorPageCustomizer　　B．BasicErrorController

C．DefaultErrorViewResolver　　D．DefaultErrorAttributes

第4章

Spring Boot 整合Thymeleaf

学习目标

- ★ 了解 Spring Boot 支持的模板引擎，能够说出 Spring Boot 支持的模板引擎有哪些
- ★ 了解 Thymeleaf 简介，能够说出 Thymeleaf 是什么和 Thymeleaf 的特点
- ★ 掌握 Thymeleaf 常用属性，能够在模板文件中正确使用 Thymeleaf 常用属性
- ★ 掌握 Thymeleaf 标准表达式，能够在模板文件中正确使用 Thymeleaf 标准表达式
- ★ 掌握图书管理案例的实现，能够使用 Spring Boot 整合 Thymeleaf 完成图书管理案例

开发 Web 应用时，为了让项目代码耦合性更低、可维护性更高，通常会采用 MVC 设计模式来实现对应的模型、视图和控制器，让业务逻辑、数据、界面显示分离。最初的 Web 应用的视图是由 HTML 元素组成的静态界面，而如今的 Web 应用更倾向于在界面中动态显示对应的数据。Spring Boot 框架提供了一些常用视图技术的整合支持，并推荐通过整合模板引擎来实现 Web 应用界面的动态化展示。下面将对 Spring Boot 支持的模板引擎进行介绍，并对其中常用的 Thymeleaf 模板引擎进行整合。

4.1 Spring Boot 支持的模板引擎

Web 开发的模板引擎可以使用户界面与业务数据分离，通过模板引擎可以将指定模板结构的模板文件和数据生成特定格式的文档。Spring Boot 框架为很多常用的模板引擎提供了整合支持，具体介绍如下。

- FreeMarker：FreeMarker 是一个基于模板生成输出文本的模板引擎，输出文本可以是 HTML 页面、电子邮件、配置文件等。FreeMarker 不是面向最终用户的，而是一个 Java 类库，是一款程序员可以嵌入所开发产品的组件。
- Groovy：Groovy 是一种基于 JVM（Java 虚拟机）的敏捷开发语言，它结合了 Python、Ruby 和 Smalltalk 的许多强大特性，Groovy 代码能够与 Java 代码很好地结合，且能扩展现有代码。Groovy 的模板引擎功能使其可以生成各种类型的格式化文件，这些格式化文件可以作为 Web 程序的视图层使用。

- Thymeleaf：Thymeleaf 是采用 Java 语言编写的模板引擎，可用于 Web 与非 Web 环境中的应用开发，浏览器可以直接打开模板文件，便于前后端联调，它是 Spring Boot 官方推荐使用的模板引擎。
- Mustache：Mustache 是轻逻辑的模板引擎（Logic-Less Templates），它是一个 JavaScript 模板，用于对 JavaScript 分离展示。Mustache 的优势在于可以应用在 JavaScript、PHP、Python、Perl 等多种编程语言中。
- Velocity：Velocity 是一个基于 Java 的模板引擎，可以通过特定的语法获取 Java 对象的数据，并将其填充到模板中，从而实现界面与 Java 代码的分离。

Spring Boot 不太支持 JSP 模板，并且没有提供对应的整合配置，这是因为使用嵌入式 Servlet 容器的 Spring Boot 应用程序对 JSP 模板有一些限制，具体如下。

- Spring Boot 默认使用嵌入式 Servlet 容器以 JAR 包方式进行项目打包部署，这种 JAR 包方式不支持 JSP 模板。
- 如果使用 Undertow 嵌入式容器部署 Spring Boot 项目，也不支持 JSP 模板。
- Spring Boot 默认提供了一个处理请求路径“/error”的统一错误处理器，用于返回具体的异常信息。使用 JSP 模板时，无法对默认的错误处理器进行覆盖，只能根据 Spring Boot 要求在指定位置定制错误页面。

4.2 Thymeleaf 基础入门

Thymeleaf 是 Spring Boot 官方推荐使用的模板引擎，它创建的模板可维护性非常高。Thymeleaf 使开发人员能够在保持设计原型不变的情况下，将程序的数据和逻辑注入模板文件中，以提高设计团队和开发团队的工作协调性，减少因理解和实现偏差产生的问题，也使得维护工作更加高效和准确。下面对 Thymeleaf 简介、Thymeleaf 常用属性、Thymeleaf 标准表达式等基础入门知识进行讲解。

4.2.1 Thymeleaf 简介

Thymeleaf 以不影响模板作为设计原型的方式，将其逻辑数据注入模板文件中，从而在开发团队中实现更强大的协作，支持 HTML、XML、TEXT、JAVASCRIPT、CSS、RAW 六种模板。Thymeleaf 模板引擎具有以下特点。

1. 动静结合

Thymeleaf 可以查看页面的静态效果和动态页面效果。如果直接使用浏览器打开模板文件，浏览器会忽略未定义的 Thymeleaf 标签属性，展示 Thymeleaf 模板文件的静态页面效果；当通过 Web 应用程序访问模板文件时，Thymeleaf 会根据获取到的数据对文件进行渲染，使页面进行动态显示。

2. 开箱即用

Thymeleaf 提供了 Spring 标准方言，以及与 Spring MVC 完美集成的可选模块，可以快速地实现表单绑定、属性编辑器、国际化等功能。

3. 多方言支持

Thymeleaf 是可扩展的模板引擎，提供了 Thymeleaf 标准和 Spring 标准方言，可以直接套用模板实现 JSTL、OGNL 表达式。如果用户希望在利用库的高级特性的同时定义自己的

处理逻辑，则可以创建自己的方言。

4. 与 Spring Boot 完美整合

Spring Boot 为 Thymeleaf 整合提供了自动配置支持，并且还为 Thymeleaf 设置了视图解析器。

4.2.2　Thymeleaf 常用属性

Thymeleaf 的 HTML 模板文件中使用的 HTML 元素语法几乎是标准的 HTML 语法，Thymeleaf 在标准 HTML 标签中增加一些格式为“ th:xxx ”的属性以实现“ 模板+数据 ”的展示，示例代码如下。

```
1  <!DOCTYPE html>
2  <html lang="en" xmlns:th="http://www.thymeleaf.org">
3  <head>
4      <meta charset="UTF-8">
5      <title>Title</title>
6  </head>
7  <body>
8  <h1 th:text="Hello Thymeleaf">Thymeleaf 静态页面</h1>
9  </body>
10 </html>
```

在上述代码中，第 8 行代码在<h1>标签中使用了一个 th:text 属性，当直接使用浏览器打开时，浏览器展示结果如图 4-1 所示。

当启动 Web 项目后，以访问 Web 应用中的资源的方式打开文件时，浏览器展示结果如图 4-2 所示。

图4-1　直接使用浏览器打开文件

图4-2　在项目中访问文件

从图 4-1 和图 4-2 可以得出，相同的页面使用不同的方式进行访问，页面效果会不同，主要原因是文件的<html>标签中声明了名称的空间，以及模板文件中<h1>标签中添加了 th:text 属性。其中，声明的名称空间可避免编辑器出现 HTML 验证错误，th:text 属性可以对<h1>标签中的文本进行替换。

除了 th:text 属性，Thymeleaf 还提供了大量的 th 属性，这些属性可以直接在 HTML 标签中使用，常用的 th 属性如表 4-1 所示。

表 4-1　Thymeleaf 常用的 th 属性

属性	作用
th:each	用于遍历，可以遍历集合、数组等对象
th:if	根据条件判断是否需要展示此标签
th:unless	与 th:if 判断相反，满足条件时不显示
th:switch	与 Java 的 switch 语句类似，通常与 th:case 配合使用，根据不同的条件展示不同的内容
th:case	条件判断，进行选择性匹配
th:object	用于绑定数据对象
th:with	定义局部变量

续表

属性	作用
th:action	表单提交地址
th:attr	动态拼接属性值
th:onclick	绑定鼠标单击事件
th:value	替换 value 属性的值
th:href	用于设定链接地址
th:src	用于替换 src 属性的值，设定链接地址
th:text	文本替换，转义特殊字符
th:utext	文本替换，不转义特殊字符
th:fragment	声明片段，类似 JSP 的 tag，用来定义一段被引用或包含的模板片段
th:insert	用于指定在模板中插入的其他模板片段，类似于 JSP 中的 include 标签，可以通过 th:fragment 属性定义要被替换进来的片段
th:replace	用于将指定位置的内容替换为指定的模板片段，可以通过 th:fragment 属性定义要被替换进来的片段
th:block	创建区域块，如果内容为空则不创建

表 4-1 中列举了 Thymeleaf 模板引擎的一些常用属性，关于其他更多的属性支持，读者在使用过程中可以查看官方文档，也可以查看开发工具的快捷提示信息。Thymeleaf 的 th 属性通常会结合 Thymeleaf 标准表达式一起使用。因此，关于 Thymeleaf 的 th 属性的具体应用会在本章后续内容中演示。

4.2.3 Thymeleaf 标准表达式

动态页面与静态页面的主要区别是动态页面中可以动态显示相关的数据，Thymeleaf 标准方言中提供了一些可以动态获取数据的标准表达式，这些标准表达式与 JSP 的 EL 功能非常相似，可以获取 Web Context 中的请求参数、请求属性、会话属性、应用属性等数据。根据语法规则，Thymeleaf 标准表达式主要分为变量表达式、选择表达式、消息表达式、链接表达式、分段表达式，下面分别对这几种表达式进行讲解。

1. 变量表达式

Thymeleaf 中使用${}包裹内容的表达式被称为变量表达式，使用变量表达式可以访问容器上下文中的变量，示例代码如下。

```
<p th:text="${title}">这是标题</p>
```

在上述示例中，使用变量表达式获取容器上下文中的变量 title，如果直接通过浏览器进行访问或者当前上下文中不存在变量 title，会显示<p>标签的默认值“ 这是标题 ”；如果通过 Web 应用程序进行访问，并且当前上下文中存在变量 title，则会将变量 title 的值赋给 th:text 属性，替换当前<p>标签中的默认文本内容，从而实现模板引擎页面的数据动态显示。

变量表达式中除了可以直接访问变量外，还可以访问变量的属性和方法，示例代码如下。

```
<p th:text="${user.name}">这是标题</p>
<p th:text="${user.info()}">这是标题</p>
```

在上述示例中，分别使用变量表达式访问变量 user 的 name 属性和变量 user 的 info()方法。

同时，Thymeleaf 为变量所在域提供了一些内置对象，具体如下。

- #ctx：上下文对象。
- #vars：上下文变量。
- #locale：上下文语言环境。
- #request：HttpServletRequest 对象，仅在 Web 应用中可用。
- #response：HttpServletResponse 对象，仅在 Web 应用中可用。
- #session：HttpSession 对象，仅在 Web 应用中可用。
- #servletContext：ServletContext 对象，仅在 Web 应用中可用。

这些内置对象都以符号#开头，通过变量表达式可以访问这些内置对象，例如，通过变量表达式访问 HttpSession 中的 name 属性，示例代码如下。

```
<p th:text="${#session.getAttribute('name')}"></p>
```

在上述代码中，通过 HttpSession 对象的 getAttribute()方法获取 name 属性的值。

除了通过上述方式操作内置对象外，在变量表达式中也可使用内置对象直接访问内置对象的属性，示例代码如下。

```
<p th:text="${session.name}"></p>
```

上述代码和使用${#session.getAttribute('name')}的方式效果一样。

为了便于开发，Thymeleaf 中还提供了一些内置的工具对象，这些工具对象就像是 Java 内置对象一样，可以直接访问对应 Java 内置对象的方法来进行各种操作，具体如下。

- #strings：字符串工具对象，常用的方法有 equals()、length()、substring()、replace()等。
- #numbers：数字工具对象，常用的方法有 formatDecimal()等。
- #bools：布尔工具对象，常用的方法有 isTrue()和 isFalse()等。
- #arrays：数组工具对象，常用的方法有 toArray()、isEmpty()、contains()等。
- #lists：List 集合工具对象，常用的方法有 toList()、size()、isEmpty()等。
- #sets：Set 集合工具对象，常用的方法有 toSet()、size()、isEmpty()、contains()等。
- #maps：Map 集合工具对象，常用的方法有 size()、isEmpty()、containsKey()、containsValue()等。
- #dates：日期工具对象，常用的方法有 format()、year()、month()、hour()等。

上述工具对象的常用方法与 Java 中的字符串、集合等对象的方法的作用一样，此处不再进行说明。

通过变量表达式可以使用工具对象，例如，在变量表达式中，使用#strings 调用 length()方法，示例代码如下。

```
<h1 th:text="${#strings.length('thymeleaf')}"></h1>
```

2．选择表达式

Thymeleaf 中使用*{}包裹内容的表达式被称为选择表达式，选择表达式与变量表达式功能基本一致，只是选择表达式计算的是绑定的对象，而不是整个环境变量映射。标签中通过“th:object”属性绑定对象后，可以在该标签的后代标签中使用选择表达式（*{...}）访问该对象中的属性，其中，“*”即代表绑定的对象。

例如，通过选择表达式访问 user 对象中的 username 属性，示例代码如下。

```
<div th:object="${user}" >
     <p th:text="*{username}">name</p>
</div>
```

在上述代码中，在 标签中使用“th:object”属性绑定对象 user；在<p>标签中使用

选择表达式访问 user 对象的 username 属性。

3. 消息表达式

Thymeleaf 中使用#{}包裹内容的表达式被称为消息表达式，消息表达式可以显示静态文本的内容，通常与 th:text 属性一起使用。消息表达式显示的内容通常是读取配置文件中信息，在#{}中指定配置文件中的 Key，则会在页面中显示配置文件中 Key 对应的 Value。

例如，通过消息表达式读取项目 Properties 配置文件中 location 属性的值，示例代码如下。

```
<p th:text="#{location}"></p>
```

在上述代码中，将 Properties 配置文件中 location 属性的值作为 th:text 属性的值。读取指定配置文件的内容时，需要先在项目的全局配置文件中通过 spring.messages.basename 属性进行文件的指定。

4. 链接表达式

Thymeleaf 中使用@{}包裹内容的表达式被称为链接表达式，表达式中包裹的内容只能写一个绝对 URL 或相对 URL 地址，如果绝对/相对 URL 地址中包含动态参数，就需要结合变量表达式进行拼接。

不管是静态资源的引用，还是 form 表单的请求，凡是链接都可以用链接表达式。链接表达式中的 URL 地址有三种写法，分别为以 http 协议名开头的绝对地址、以“/”开头的根相对地址、不以“/”开头的相对地址。其中，Thymeleaf 会将开头的“/”解析为当前工程的上下文路径 ContextPath，而浏览器会自动为其添加“http://主机名:端口号”。

例如，使用链接表达式引入 CSS 样式表，示例代码如下。

```
<link rel="stylesheet" th:href="@{/css/bootstrap.css}">
```

在上述代码中，通过链接表达式引入项目 css 文件夹下的 bootstrap.css。

5. 分段表达式

Thymeleaf 中使用～{}包裹内容的表达式被称为分段表达式，分段表达式用于在模板页面中引用其他的模板片段，该表达式支持以下两种语法结构。

～{templatename::fragmentname}。

～{templatename::#id}。

上述语法格式中的参数说明如下。

- templatename：模板名，Thymeleaf 默认会在/resources/templates/目录下根据模板名查找对应的模板文件，然后根据对应的模板文件进行解析。
- fragmentname：片段名，Thymeleaf 通过 th:fragment 声明定义片段，即 th:fragment="fragmentname"。
- #id：HTML 的 id 选择器。

分段表达式最常见的用法是使用 th:insert 或 th:replace 属性插入片段，示例代码如下。

```
<div th:insert="～{thymeleafDemo::title}"></div>
```

在上述代码中，使用 th:insert 属性将 title 片段模板插入该 标签中。thymeleafDemo 为模板名称，Thymeleaf 会自动查找“/resources/templates/”目录下 thymeleafDemo 模板中声明的名称为 title 的片段。

4.3　案例：图书管理

早在一千九百多年前，中国就发明了造纸术，纸的发明与改进，促进了书籍的社会生产；大约在一千三百年前，中国发明了雕版印刷术；一千多年前，又发明了活字印刷术。这一系列重大发明，不但使书籍的社会生产跨进了一个新的时代，也使人类文明跨进了一个新的时代。这是中华民族为人类文明进步作出的伟大贡献。随着社会的进步和发展，书籍越来越多，为了对书籍进行更好的共享，图书管理系统应运而生。

通过前面的学习，相信读者对 Thymeleaf 的作用、常用属性和标准表达式有了一定的了解，为了加深读者对 Thymeleaf 相关知识的理解，下面通过一个图书管理案例演示 Spring Boot 整合 Thymeleaf 的应用，案例的页面效果如图 4-3 所示。

图4-3　案例页面效果

1. 任务需求

本案例要求实现以下需求。

（1）项目使用 Spring Boot 整合 Thymeleaf，项目展示的页面效果全部通过 Thymeleaf 的模板文件实现。

（2）查询所有图书。访问 http://localhost:8080/book/list 时，查询所有图书，并展示在页面中。

（3）选择性显示按钮。当 Session 中存在用户角色为“ADMIN”时，显示“新增”按钮，否则不显示该按钮。

（4）按条件查询图书。单击“查询”按钮时，根据搜索框中的查询条件查询对应的图书信息。

（5）借阅操作。当图书状态为可借阅时，对应的“借阅”按钮为可用状态，并且单击“借阅”按钮时，将当前申请借阅图书的编号异步发送到后台。

2. 任务实现

（1）Spring Boot 整合 Thymeleaf

Spring Boot 为 Thymeleaf 提供了一系列默认配置，Spring Boot 项目中只要引入 Thymeleaf 的启动器，项目启动时就会自动进行相应配置，因此 Spring Boot 整合 Thymeleaf 只需在创建的 Spring Boot 项目中添加 Thymeleaf 启动器即可。

根据创建 Spring Boot 项目的方式，在 Spring Boot 项目中添加 Thymeleaf 启动器有两种方法，具体如下。

使用 Spring Initializr 方式构建 Spring Boot 项目时，在 Spring Boot 场景依赖选择界面

添加 Thymeleaf 依赖。在搜索框中查找对应的依赖，勾选所需的依赖后，项目创建成功后会自动添加对应的依赖，如图 4-4 所示。

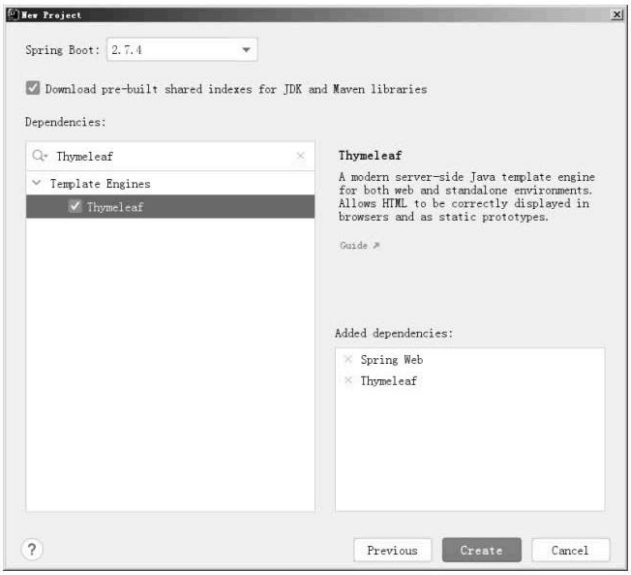

图4-4　添加Thymeleaf依赖

图 4-4 中添加了 Spring Web 和 Thymeleaf 的依赖。

使用 Maven 方式构建 Spring Boot 项目后 ,在 pom.xml 文件中引入 Thymeleaf 启动器。同时项目需要使用 Spring MVC，具体如下。

```
<dependencies>
<!--Thymeleaf 启动器-->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

这两种整合方式没有区别，读者可以根据自己的习惯或者当前主机的联网状态自行选择其一即可。

Spring Boot 整合 Thymeleaf 后，项目启动时会加载 ThymeleafProperties 类中提供的默认配置信息，该默认配置中指定了模板文件的默认路径为 classpath:/templates、模板文件的默认后缀为.html 等信息。默认配置信息比较多，在此就不一一列举了，通常来说开发人员不用对这些默认配置进行修改。

（2）创建实体类

图书管理页面中包含两个实体对象，分别为用户实体和图书实体，用户实体主要用于根据用户角色选择性展示“ 新增 ”按钮，图书实体主要用于展示封装页面的图书信息。当前还没有学习 Spring Boot 和持久层技术的整合，因此选择通过一个类封装一些图书数据，

页面展示的图书信息都从这个类中获取数据，需要创建用户类、图书类和数据类，具体如文件 4-1～文件 4-3 所示。

文件 4-1　User.java

```
public class User  {
    private String name;      //用户名称
    private String role;      //用户角色
    public User(String name, String role) {
        this.name = name;
        this.role = role;
    }
   //……setter/getter 方法
}
```

文件 4-2　Book .java

```
public class Book  {
    private Integer id;       //图书编号
    private String name;      //图书名称
    private String author;    //图书作者
    private String press;     //图书出版社
    private String status;    //图书状态
    public Book(Integer id, String name, String author,String press,
        String status) {
        this.id = id;
        this.name = name;
        this.press = press;
        this.author = author;
        this.status = status;
    }
 //……setter/getter 方法
}
```

文件 4-3　Data.java

```
import java.util.ArrayList;
public class Data {
    public static ArrayList<Book> getData(){
        ArrayList<Book> books=new ArrayList<>();
        books.add(new Book(1,"楚辞","屈原","中国文联出版社","0"));
        books.add(new Book(2,"纳兰词","纳兰性德","中国文联出版社","1"));
        books.add(new Book(3,"西游记","吴承恩","中国文联出版社","2"));
        return books;
    }
}
```

（3）创建控制器类

通常模板文件中展示的数据都是在服务器端查询数据库后传递过来的，本案例没有整合持久层技术，在控制层中获取 Data 类中的数据进行返回。任务要求中需要查询所有图书、按条件查询图书，以及获取借阅图书的编号，因此可以创建控制器类，在控制器类中创建对应的方法接收和响应页面发送的请求，具体如文件 4-4 所示。

文件 4-4　BookController.java

```
import com.itheima.chapter04.entity.Book;
import com.itheima.chapter04.entity.Data;
```

```
3  import com.itheima.chapter04.entity.User;
4  import org.springframework.stereotype.Controller;
5  import org.springframework.ui.Model;
6  import org.springframework.web.bind.annotation.PathVariable;
7  import org.springframework.web.bind.annotation.RequestMapping;
8  import javax.servlet.http.HttpSession;
9  import java.util.ArrayList;
10 @Controller
11 @RequestMapping("book")
12 public class BookController {
13     //获取所有图书信息
14     ArrayList<Book> books = Data.getData();
15     /**
16      * 查询所有图书
17      */
18     @RequestMapping("list")
19     public String findBook(Model model, HttpSession session){
20         session.setAttribute("user",new User("zhangsan","ADMIN"));
21         model.addAttribute("books",books);
22         return "books";
23     }
24     /**
25      *按条件查询图书
26      */
27     @RequestMapping("search")
28     public String searchBook(Book book,Model model,HttpSession session){
29         ArrayList<Book> bs=new ArrayList<>();
30         String bname=book.getName();
31         String bauthor=book.getAuthor();
32         if(bname.isEmpty()&&bauthor.isEmpty()){
33             bs=books;
34         }else{
35             for (Book b : books) {
36                 if((!bname.isEmpty()&&b.getName().contains(bname))||
37                     (!bauthor.isEmpty()&&b.getAuthor().contains(bauthor))){
38                     bs.add(b);
39                 }
40             }
41         }
42         session.setAttribute("user",new User("zhangsan","ADMIN"));
43         model.addAttribute("books",bs);
44         return "books";
45     }
46     /**
47      *获取借阅图书的编号
48      */
49     @RequestMapping("find/{id}")
50     public String findBook(@PathVariable("id") Integer id){
51         System.out.println("申请借阅图书的 id: "+id);
52         return "books";
53     }
```

```
54 }
```

在上述代码中，第 14 行代码获取了 Data 类中预备的图书信息，在本案例中为所有的图书信息。第 18～23 行代码中定义了 findBoos()方法用于接收和响应查询所有图书的请求，其中第 20 行代码将角色为 ADMIN 的用户信息存入 Session 中，第 21～22 行代码将获取到的所有图书信息存放到 model 对象中返回。

第 27～45 行代码中定义了 searchBook()方法用于接收和响应按条件查询图书信息的请求，其中第 32 行代码判断请求中的查询条件是否都为空，如果都为空，则返回所有的图书信息；如果不是都为空，则根据查询条件返回对应的图书信息。

第 49～54 行代码中定义了 findBook()方法用于接收和响应图书借阅的请求，本案例只演示是否成功接收到异步请求，因此只将请求中的图书编号输出在控制台中。

（4）创建模板文件

将教材提供的样式文件导入 resources\static 目录下。Spring Boot 项目默认查找 classpath 中 templates 文件夹下的模板文件，并且默认不允许直接访问 templates 下的文件。控制层中的方法在处理请求后跳转回名为 books 的视图。为此，在 resources\templates 目录下创建名为 books 的 HTML 文件作为模板文件，将模板文件中通过 Thymeleaf 的属性和标准表达式引入样式文件，并获取数据进行展示，具体如文件 4-5 所示。

文件 4-5　books.html

```
1  <html lang="en" xmlns:th="http://www.thymeleaf.org">
2  <head>
3      <meta charset="utf-8">
4      <title>图书管理</title>
5      <link rel="stylesheet" th:href="@{/css/bootstrap.css}">
6      <link rel="stylesheet" th:href="@{/css/AdminLTE.css}">
7      <link rel="stylesheet" th:href="@{/css/pagination.css}">
8      <script th:src="@{/js/jquery.min.js}"></script>
9  </head>
10 <body>
11 <div class="box-body">
12     <div class="pull-left"
13          th:if="${#session.getAttribute('user').role =='ADMIN'}">
14         <div class="form-group ">
15             <button type="button" class="btn btn-default"> 新增 </button>
16         </div>
17     </div>
18     <!--数据搜索 -->
19     <div class="pull-right">
20         <div class="has-feedback">
21             <form th:action="@{/book/search}" method="post">
22                 图书名称：<input name="name">    
23                 图书作者：<input name="author">    
24                 <input class="btn btn-default" type="submit" value="查询">
25             </form>
26         </div>
27     </div>
28     <div class="table-box">
29         <!-- 数据表格 -->
```

```
30      <table id="dataList" class="table table-bordered
31            table-striped table-hover text-center">
32         <thead>
33         <tr>
34            <th >图书名称</th>
35            <th >图书作者</th>
36            <th >出版社</th>
37            <th >图书状态</th>
38            <th >操作</th>
39         </tr>
40         </thead>
41         <tbody>
42         <th:block th:each="book : ${books}">
43            <tr>
44               <td th:text="${book.name}"/></td>
45               <td th:text="${book.author}"></td>
46               <td th:text="${book.press}"></td>
47               <td>
48                  <th:block th:if="${book.status == '0'}">
49                     可借阅
50                  </th:block>
51                  <th:block th:if="${book.status =='1'}">
52                     借阅中
53                  </th:block>
54                  <th:block th:if="${book.status =='2'}">
55                     归还中
56                  </th:block>
57               </td>
58               <td class="text-center">
59                  <button th:if="${book.status =='0'}" type="button"
60                          class="btn bg-olive btn-xs"
61                          th:onclick="findBookById([[${book.id}]]);">
62                          借阅
63                  </button>
64                  <button th:if="${book.status =='1' ||
65                         book.status =='2'}" type="button"
66                         class="btn bg-olive btn-xs"
67                         disabled="true">借阅
68                  </button>
69               </td>
70            </tr>
71         </th:block>
72         </tbody>
73      </table>
74   </div>
75 </div>
76 </body>
77 <script>
78    function findBookById(id) {
79       $.get("/book/find/" + id)
80    }
```

```
81 </script>
82 </html>
```

在上述代码中 ,第 5～8 行代码分别使用 th:href 属性和 th:src 属性通过链接 URL 表达式引入 resources 下的 CSS 文件和 JavaScript 文件。第 12～17 行代码用于判断 Session 中 user 属性的 role 属性是否为 ADMIN，如果是，则模板文件渲染后显示“ 新增 ”按钮。

第 21～25 行代码在 标签中定义了一个表单，表单中包含按条件查询图书的文本框和按钮，其中使用 th:action 属性和链接 URL 表达式指定表单提交的地址。

第 30～73 行代码定义了一个用于展示图书信息列表的表格，其中第 42 行代码使用 th:each 属性迭代上下文环境中名称为 books 的对象，每次迭代出的对象临时存放在名称为 book 的对象中；第 44～46 行代码依次使用 th:text 属性和变量表达式将 book 对象中属性为 name、author、press 的值设置在对应的单元格中；第 47～57 行代码使用 th:if 属性判断 book 的 status 属性的值，以在单元格中显示对应的借阅状态；第 59～68 行代码根据图书的借阅状态设置操作按钮是否可用 ,并为可用状态的按钮绑定鼠标单击事件 ,单击时执行 findBookById()方法。

第 78～79 行代码定义了 findBookById()方法，用于将图书编号异步发送到服务器端。

（5）测试图书管理

启动项目，在浏览器中访问 http://localhost:8080/book/list 查询所有图书信息，效果如图 4-5 所示。

图4-5　查询所有图书信息

从图 4-5 可以看出，控制器将所有图书信息响应给 books.html 模板文件后，模板引擎根据模板和数据进行渲染，最终页面成功展示对应的图书信息，并根据用户角色和图书借阅状态选择性显示了“ 新增 ”按钮和“ 借阅 ”按钮。说明页面成功导入样式文件，各标签上的 th 属性也成功被渲染。

在图 4-5 所示界面中输入查询条件后单击“ 查询 ”按钮，按条件查询图书信息，以查询图书名称中包含“ 辞 ”字的图书信息为例，效果如图 4-6 所示。

从图 4-6 可以看出 ,页面展示了图书名称中包含“ 辞 ”的图书信息 ,说明表单的 th:action 属性定义成功。

在图 4-6 所示界面中单击“ 借阅 ”按钮，此时 IDEA 控制台中输出如图 4-7 所示。

从图 4-7 可以看出 ,单击“ 借阅 ”按钮时 ,页面成功发送请求到控制器 ,说明 JavaScript 文件导入成功，并且按钮上的鼠标单击事件绑定成功。

至此，基于 Spring Boot、Spring MVC 和 Thymeleaf 的图书管理案例已经完成。

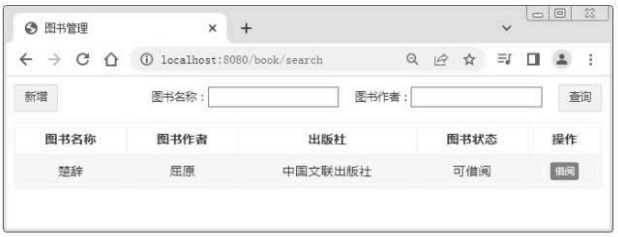

图4-6 按条件查询图书信息

图4-7 控制台输出信息

4.4 本章小结

本章主要对 Spring Boot 整合 Thymeleaf 进行了讲解。首先讲解了 Spring Boot 支持的模板引擎；然后讲解了 Thymeleaf 基础入门；最后通过图书管理案例演示 Spring Boot 整合 Thymeleaf，以及 Thymeleaf 常用属性和标准表达式的应用。通过本章的学习，希望大家可以对 Spring Boot 整合 Thymeleaf 有所了解，为后续更深入学习 Spring Boot 做好铺垫。

4.5 本章习题

一、填空题

1．Thymeleaf 的_____属性根据条件判断是否需要展示此标签。

2．Thymeleaf 中使用_____包裹内容的表达式被称为变量表达式。

3．使用 Thymeleaf_____表达式可以访问容器上下文中的变量。

4．Thymeleaf 的选择表达式计算的是_____。

5．Spring Boot 整合 Thymeleaf 后，模板文件的默认后缀为_____。

二、判断题

1．Thymeleaf 是采用 Java 语言编写的模板引擎。（　　）

2．Thymeleaf 的 th 属性可以直接在 HTML 标签中使用。（　　）

3．Thymeleaf 的 th:each 属性用于遍历，可以遍历集合、数组等对象。（　　）

4．Thymeleaf 默认会在/resources/templates/目录下根据模板名查找对应的模板文件。（　　）

5．链接表达式中包裹的内容可以是绝对 URL 或相对 URL 地址。（　　）

三、选择题

1．下列选项中，关于 Thymeleaf 的描述错误的是（　　）。

A．Thymeleaf 是采用 Python 语言编写的模板引擎。

B．Thymeleaf 可用于非 Web 环境中的应用开发。

C．Thymeleaf 浏览器可以直接打开模板文件。

D．Thymeleaf 是 Spring Boot 官方推荐使用的模板引擎。

2．下列选项中，关于 Thymeleaf 的特点描述错误的是（　　）。

A．Thymeleaf 提供了 Spring 标准方言。

B．如果直接使用浏览器打开模板文件，浏览器不解析未定义的 Thymeleaf 标签属性，

导致页面打开错误。

C．Thymeleaf 可以创建自己的方言。

D．Spring Boot 为 Thymeleaf 整合提供了自动配置支持。

3．下列选项中，用于根据条件判断是否需要展示当前标签的 Thymeleaf 属性是(　　)。

A．th:each　　B．th:if　　C．th:value　　D．th:object

4．下列选项中，关于 Thymeleaf 的内置对象描述错误的是（　　）。

A．#request 对应 HttpServletRequest 对象。

B．#response 对应 HttpServletResponse 对象。

C．#session 对应 HttpSession 对象。

D．#servletContext 对应上下文变量。

5．下列选项中，关于 Thymeleaf 的表达式描述错误的是（　　）。

A．变量表达式中也使用内置对象直接访问内置对象的属性。

B．选择表达式是对整个应用环境的变量进行映射。

C．在#{}中指定配置文件中的 Key，则会在页面中显示配置文件中 Key 对应的 Value。

D．Thymeleaf 会将链接表达式中开头的“ / ”,解析为当前工程的上下文路径 ContextPath。

第5章

Spring Boot数据访问

学习目标

★ 熟悉 Spring Data 概述，能够说出 Spring Data 的项目结构和 Spring Data 的常用核心子接口

★ 了解 Spring Data JPA 概述，能够说出使用 Spring Data JPA 进行数据访问的逻辑

★ 掌握 Spring Data JPA 快速入门，能够根据方法命名规则定义的方法、JPQL，以及原生 SQL 的方式操作数据库中的数据

★ 掌握 Spring Boot 整合 Spring Data JPA，能够整合 Spring Boot 和 Spring Data JPA，并使用 Spring Data JPA 进行基本的增删改查

★ 了解 MyBatis-Plus 概述，能够说出 MyBatis-Plus 的特性

★ 掌握 MyBatis-Plus 快速入门，能够使用通用 Mapper、通用 Service，以及条件构造器操作数据库中的数据

★ 掌握 Spring Boot 整合 MyBatis-Plus，能够整合 Spring Boot 和 MyBatis-Plus，并使用 MyBatis-Plus 进行基本的增删改查

★ 熟悉 Redis 快速入门，能够说出 Redis 的概念和优点、安装和启动 Redis 的方法，以及 Redis 支持的数据类型

★ 掌握 Spring Data Redis 快速入门，能够说出 Spring Data Redis 的特性，以及应用 Spring Data Redis 的常见操作

★ 掌握 Spring Boot 整合 Redis，能够整合 Spring Boot 和 Redis，并使用 Spring Data Redis 向 Redis 中存储和读取数据

一般情况下，应用程序中的数据都会使用数据库（Data Base，DB）进行存储和管理，然后在应用程序中对数据库中的数据进行访问操作。Spring Boot 在简化项目开发以及实现自动化配置的基础上，对常见的数据层解决方案提供了非常好的支持，用户只需要进行简单的配置，就可以使用数据库访问技术对数据库进行数据访问。下面将对 Spring Boot 项目中访问数据库的常用技术进行讲解。

5.1　Spring Data 概述

Spring Boot 默认采用整合 Spring Data 的方式统一处理数据访问层，Spring Data 是 Spring 提供的开源框架，旨在统一和简化对各种类型数据库的持久化存储。Spring Data 为大量的关系型数据库和非关系型数据库提供了数据访问的方案，并且提供了基于 Spring 的数据访问模型，同时保留了各存储系统的特殊性。

Spring Data 为开发者提供了一套统一的 API，开发者可用同一套 API 操作不同存储系统中的数据，保持代码结构的一致性，从而大幅减少开发工作量，提高开发效率。Spring Data 抽象项目结构如图 5-1 所示。

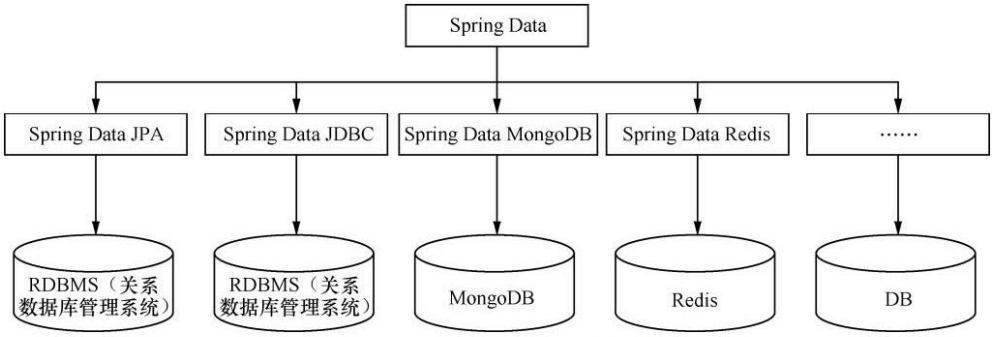

图5-1　Spring Data抽象项目结构

从逻辑关系上说，Spring Data 相当于 Spring Data 各个子项目的抽象层，而 Spring Data 的子项目是具体实现。Spring Data 中引入各种数据访问模板和统一的 Repository 接口，将应用程序从底层数据访问的具体实现中抽离，简化了数据访问层的操作。

Spring Data 提供了一套统一的 Repository 接口实现方式，包括基本的增删改查操作、条件查询、排序查询、分页查询等。当数据访问对象需要进行增删改查、排序查询和分页查询等操作时，开发者仅需要按照一定的规范声明接口即可，不需要实现具体的查询方法。Spring Data 会根据底层数据存储系统，在运行时自动实现真正的查询方法，执行查询操作，返回结果数据集。

Repository 接口是一个空接口，它的几个子接口扩展了一些常用的功能，其中常见的核心接口之间的继承关系如图 5-2 所示。

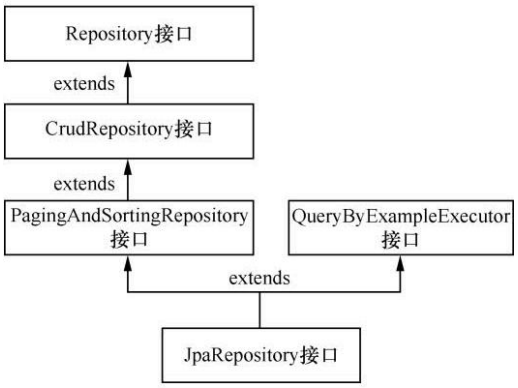

图5-2　常见的核心接口之间的继承关系

下面对图 5-2 中的接口进行说明，具体如下。

- Repository 接口：Repository 是一个空接口，用于标示。若定义的接口继承了 Repository 接口，则该接口会被 IoC 容器识别为一个 Repository Bean 纳入到 IoC 容器中。
- CrudRepository 接口：继承 Repository 接口，提供了各种增删改查方法，继承该接口的 DAO 组件不用开发者编写任何代码就可执行各种增删改查数据操作。
- PagingAndSortingRepository 接口：继承 CrudRepository 接口，增加了分页和排序的方法。
- QueryByExampleExecutor 接口：是进行条件封装查询的顶级父接口，允许通过 Example 实例执行复杂条件查询。
- JpaRepository 接口：JpaRepository 接口同时继承了 PagingAndSortingRepository 接口和 QueryByExampleExecutor 接口，并重写了一些查找和删除的方法。

5.2 Spring Boot 整合 Spring Data JPA

Spring Data 作为 Spring 全家桶中重要的一员，在 Spring 项目全球使用市场份额排名中多次居前位，而在 Spring Data 子项目的使用份额排名中，Spring Data JPA 也一直名列前茅。Spring Boot 为 Spring Data JPA 提供了启动器，使 Spring Data JPA 在 Spring Boot 项目中的使用更加便利。下面将对 Spring Data JPA 的相关知识，以及 Spring Boot 整合 Spring Data JPA 进行讲解。

5.2.1 Spring Data JPA 概述

对象关系映射（Object Relational Mapping，ORM）框架在运行时可以参照映射文件的信息，把对象持久化到数据库中，可以解决面向对象与关系数据库存在的互不匹配的现象，常见的 ORM 框架有 Hibernate、OpenJPA 等。ORM 框架的出现，使开发者从数据库编程中解脱出来，把更多的精力放在业务模型和业务逻辑上，但各 ORM 框架之间的 API 差别很大，使用了某种 ORM 框架的系统会严重受限于该 ORM 的标准，基于此，Sun 公司提出了 JPA（Java Persistence API，Java 持久化 API）。

JPA 是 Sun 官方提出的 Java 持久化规范，用于描述对象和关系表的映射关系，并将运行期的实体对象持久化到数据库中。JPA 规范本质上是一套规范，它提供了一些编程的 API，但具体实现则由服务厂商来提供基于 JPA 的数据访问，如图 5-3 所示。

Spring Data JPA 是 Spring 基于 ORM 框架和 JPA 规范封装的一套 JPA 应用框架，在 JPA 基础上提供更高层次的抽象，可以减少很多模板代码，可以使开发者仅用极简的代码实现对数据的访问和操作。Spring Data JPA 默认使用 Hibernate 实现 JPA。

应用程序中使用 Spring Data JPA 进行数据访问时，Java 业务层调用 Spring Data JPA 二次封装提供的 Repository 接口，接着 Repository 接口的具体实现会基于 JPA 标准 API 进行处理，JAP 的标准 API 会基于 Hibernate 提供的 JPA 具体实现进行进一步处理，接着 Hibernate 底层基于 JDBC 标准 API 完成与实际 DB 之间的请求交互。Spring Data JPA 整体处理逻辑如图 5-4 所示。

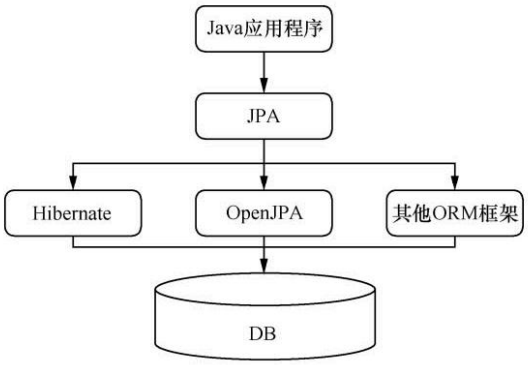

图5-3　基于JPA的数据访问

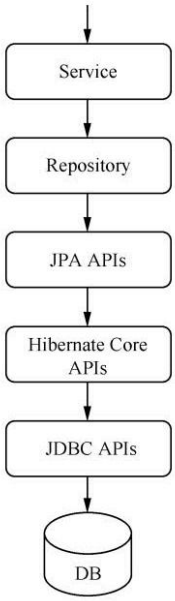

图5-4　Spring Data JPA整体处理逻辑

简而言之，JPA 指明了持久化、读取和管理 Java 对象映射到数据库表时的规范。Spring Data JPA 是 Spring Data 对 JPA 封装之后的产物，可在 JPA 基础上提供更高层次的抽象。使用 Spring Data JPA 操作数据库，只需要按照框架的规范提供接口，不需要在接口中定义方法和为接口提供实现类，就可以对数据库中的数据进行增删改查操作。

5.2.2　Spring Data JPA 快速入门

Spring Data JPA 提供了很多模板代码，易于扩展，可以大幅提高开发效率，使开发者用极简的代码即可实现对数据的访问。基于 Spring Data JPA 的规范可以提升数据访问层的开发效率和灵活性，开发人员在使用 Spring Data JPA 时，应基于 Spring Data JPA 的规范，同时也需要遵守团队的代码规范和风格，保证代码清晰易读，从而开发高质量的程序。

使用 Spring Data JPA 可以通过 Repository 接口中的方法对数据库中的数据进行增删改查，也可以根据方法命名规则定义的方法、Java 持久化查询语言（Java Persistence Query Language，JPQL），以及原生 SQL 进行操作，下面对 Spring Data JPA 提供的这些基本功能进行讲解。

1. 父接口的方法

在使用 Spring Data JPA 进行数据操作时，通常会选择继承 JpaRepository 接口，JpaRepository 接口可以对其他接口的方法的返回值进行适配处理。如果自定义接口继承了 JpaRepository 接口，则可以直接使用 JpaRepository 接口提供的方法，如图 5-5 所示。

图5-5　JpaRepository接口的方法

图 5-5 中展示的 JpaRepository 接口的方法默认包含了一些常用的增删改查方法。除了可以使用图 5-5 中所示的方法外，还可以使用 JpaRepository 继承的其他接口的常规增删改查和分页查询、排序查询等方法，从而满足开发时绝大部分场景的需求。

2. 根据方法命名规则定义方法

Spring Data 中按照框架的规范自定义了 Repository 接口，除了可以使用接口提供的默认方法外，还可以按特定规则来定义查询方法，只要这些查询方法的方法名遵守特定的规则，不需要提供方法实现体，Spring Data 就会自动为这些方法生成查询语句。Spring Data 对这种特定的查询方法的定义规范如下。

- 以 find、read、get、query、count 开头。
- 涉及查询条件时，条件的属性使用条件关键字连接，并且条件属性的首字母大写。

从上述规范可知，根据方法命名规则查询时，有时候需要使用关键字对查询条件的属性进行连接。Spring Data JPA 中根据方法命名规则定义方法所支持的关键字如表 5-1 所示。

表 5-1 根据方法命名规则定义方法所支持的关键字

关键字	方法名示例	对应的 JPQL 片段
And	findByLastnameAndFirstname( )	… where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname( )	… where x.lastname = ?1 or x.firstname = ?2
Is，Equals	findByFirstname，findByFirstnameIs，findByFirstnameEquals( )	… where x.firstname = ?1
Between	findByStartDateBetween( )	… where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan( )	… where x.age < ?1
LessThanEqual	findByAgeLessThanEqual( )	… where x.age <= ?1
GreaterThan	findByAgeGreaterThan( )	… where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual( )	… where x.age >= ?1
After	findByStartDateAfter( )	… where x.startDate > ?1
Before	findByStartDateBefore( )	… where x.startDate < ?1
IsNull	findByAgeIsNull( )	… where x.age is null
IsNotNull	findByAgeIsNotNull( )	… where x.age is not null
NotNull	findByAgeNotNull( )	… where x.age not null
Like	findByFirstnameLike( )	… where x.firstname like ?1
NotLike	findByFirstnameNotLike( )	… where x.firstname not like ?1
StartingWith	findByFirstnameStartingWith( )	… where x.firstname like ?1 (绑定参数 %)
EndingWith	findByFirstnameEndingWith( )	… where x.firstname like ?1 (绑定参数 %)
Containing	findByFirstnameContaining( )	… where x.firstname like ?1 (绑定参数 %)
OrderBy	findByAgeOrderByLastnameDesc( )	… where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot( )	… where x.lastname <> ?1
In	findByAgeIn(Collection<Age> ages)	… where x.age in ?1

续表

关键字	方法名示例	对应的 JPQL 片段
NotIn	findByAgeNotIn(Collection<Age> ages)	… where x.age not in ?1
True	findByActiveTrue( )	… where x.active = true
False	findByActiveFalse( )	… where x.active = false
IgnoreCase	findByFirstnameIgnoreCase( )	… where UPPER(x.firstame) = UPPER(?1)

3. JPQL

使用 Spring Data JPA 提供的查询方法已经可以满足大部分应用场景的需要，但是有些业务需要更灵活的查询条件，这时就可以使用@Query 注解结合 JPQL(Java Persistence Query Language，Java 持久化查询语言）方式来完成查询。JPQL 是 JPA 中定义的一种查询语言，此种语言旨在让开发者忽略数据库表和表中的字段，而关注实体类和实体类中的属性。JPQL 语句的写法和 SQL 语句的写法十分类似，但是要把查询的表名换成实体类名称，把表中的字段名换成实体类的属性名称。

JPQL 支持命名参数和位置参数两种查询参数。使用命名参数时，在方法的参数列表中，使用@Param 注解标注参数的名称，在@Query 注解的查询语句中，使用“ :参数名称 ” 匹配参数名称。使用位置参数时，在@Query 注解的查询语句中，使用“ ?位置编号的数值 ” 匹配参数，查询语句中参数标注的编号需要与方法的参数列表中参数的顺序依次对应。在 JPQL 中使用命名参数和位置参数的示例代码如下。

```
//命名参数绑定
@Query("from Book b where b.author=:author and b.name=:name")
List<Book> findByCondition1(@Param("author") String author,
 @Param("name") String name);
//位置参数绑定
@Query("from Book b where b.author=?1 and b.name=?2")
List<Book> findByCondition2(String author, String name);
```

JPQL 中使用 like 模糊查询、排序查询、分页查询子句时，其用法与 SQL 中的用法相同，区别在于 JPQL 处理的类的实例不同，示例代码如下。

```
//like 模糊查询
@Query("from Book b where b.name like %:name%")
 List<Book> findByCondition3(@Param("name") String name);
//排序查询
 @Query("from Book b where b.name like %:name% order by id desc")
List<Book> findByCondition4(@Param("name") String name);
//分页查询
@Query("from Book b where b.name like %:name%")
Page<Book> findByCondition5(Pageable pageable, @Param("name") String name);
```

JPQL 中除了可以使用字符串和基本数据类型的数据作为参数外，还可以使用集合和 Bean 作为参数，传入 Bean 进行查询时可以在 JPQL 中使用 SpEL 表达式接收变量，示例代码如下。

```
//传入集合参数查询
@Query("from Book b where b.id in :ids")
List<Book> findByCondition6(@Param("ids") Collection<String> ids);
```

```
//传入 Bean 进行查询（使用 SpEL 表达式）
@Query("from Book b where b.author=:#{#Book.author} and " +
" b.name=:#{#Book.name}")
Book findByCondition7(@Param("Book") Book Book);
```

4. 原生 SQL

如果出现非常复杂的业务情况，导致 JPQL 和其他查询都无法实现对应的查询，需要自定义 SQL 进行查询时，可以在@Query 注解中定义该 SQL。@Query 注解中定义的是原生 SQL 时，需要在注解使用 nativeQuery=true 指定执行的查询语句为原生 SQL，否则会将其当作 JPQL 执行，示例代码如下。

```
@Query(value="SELECT * FROM book WHERE id = :id",nativeQuery=true)
Book findByCondition8(@Param("id") Integer id);
```

小提示：

使用@Query 注解可以执行 JPQL 和原生 SQL 查询，但是@Query 注解无法进行 DML（数据操纵语言，主要语句有 INSERT、DELETE 和 UPDATE）操作，如果需要更新数据库中的数据，需要在对应的方法上标注@Modifying 注解，以通知 Spring Data 当前需要进行的是 DML 操作。需要注意的是，JPQL 只支持 DELETE 和 UPDATE 操作，不支持 INSERT 操作。

5.2.3 整合 Spring Data JPA

通过前面的学习，读者对 Spring Data JPA 有了基本的了解，下面将 Spring Boot 和 Spring Data JPA 进行整合，进一步演示 Spring Data JPA 在 Spring Boot 项目中的基本使用，具体如下。

1. 创建项目

在 IDEA 中创建 Spring Boot 项目 chapter05，读者可以根据自己当前情况选择使用 Spring Initializr 方式或者 Maven 方式进行创建，这里使用 Maven 方式创建项目。

2. 配置依赖

在项目 chapter05 的 pom.xml 文件中配置 Spring Boot 整合 Spring Data JPA 的依赖，包括 Spring Boot 父工程的依赖、MySQL 驱动依赖和 Spring Data JPA 的启动器依赖，具体如文件 5-1 所示。

文件 5-1 pom.xml

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4           xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5  https://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <parent>
8        <groupId>org.springframework.boot</groupId>
9        <artifactId>spring-boot-starter-parent</artifactId>
10       <version>2.7.4</version>
11       <relativePath/>
12    </parent>
13    <groupId>com.itheima</groupId>
14    <artifactId>chapter05</artifactId>
15    <version>0.0.1-SNAPSHOT</version>
```

```
    <name>chapter05</name>
    <description>chapter05</description>
    <properties>
        <java.version>11</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
        </dependency>
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

3. 设置配置信息

使用 Spring Data JPA 需要操作数据库，所以需要在项目中设置一些与数据库连接相关的配置信息。Spring Boot 的自动装配提供了数据库连接的一些默认配置，例如数据源连接池。Spring Boot 2.x 版本默认使用 HikariCP 作为数据源连接池，如果没有显示指定使用其他数据源连接池，项目启动后会自动使用 HikariCP 数据源连接池获取数据库连接。

引入 Spring Data JPA 的启动器后，Spring Boot 会自动装配对于 JPA 的默认配置，例如是否打印运行时的 SQL 语句和参数信息、是否根据实体自动建表等。

本项目选择采用默认的数据源连接池，JPA 的配置信息只修改打印运行时的 SQL 语句，其他都采用默认的配置信息。

在项目的 resources 目录下创建 application.yml 文件，在该文件中指定数据库连接信息和 JPA 的配置信息，具体如文件 5-2 所示。

文件 5-2　application.yml

```
spring:
  datasource:
```

```
3     url: "jdbc:mysql://localhost:3306/springbootdata?\
4     characterEncoding=utf-8&serverTimezone=Asia/Shanghai"
5     username: root
6     password: root
7   jpa:
8     show-sql: true
```

在上述代码中，第2～6行代码配置了数据库的连接信息，第7～8行代码设置使用Spring Data JPA 操作数据库时，将对应的 SQL 输出到控制台中。

4. 创建实体类

通过 JPA 可以简单高效地管理 Java 实体类和关系数据库的映射，通常每个实体类与数据库中的单个关系表相关联，每个实体的实例表示数据库表格中的某一行记录。在项目的 java 目录下创建包 com.itheima.chapter05.entity，并在该包下创建实体类 Book，具体如文件 5-3 所示。

文件 5-3 Book.java

```
1  import javax.persistence.*;
2  @Entity
3  @Table(name="book")
4  public class Book {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Integer id;       //图书编号
8     @Column(name="name")
9     private String name;      //图书名称
10    private String author;    //图书作者
11    private String press;     //图书出版社
12    private String status;    //图书状态
13     //setter/getter 方法，以及 toString()方法
14 }
```

在上述代码中，第 2 行代码使用@Entity 注解表示定义该类为一个实体类；第 3 行代码使用@Table 注解指定当前实体类对应的数据表名称，建立实体类和数据表的映射关系；第 5 行代码使用@Id 注解声明当前属性对应数据表的主键；第 6 行代码通过 @GeneratedValue (strategy = GenerationType.IDENTITY)指定主键的生成策略为数据库自动生成；第 8 行代码使用@Column 注解声明当前属性和数据表字段的对应关系，如果属性名称和字段名称一致，可以省略。

5. 自定义 Repository 接口

在 java 目录下创建包 com.itheima.chapter05.dao，在该包下自定义接口 BookRepository 继承 JpaRepository 接口，并在 BookRepository 接口中添加操作数据库中图书信息的方法，具体如文件 5-4 所示。

文件 5-4 BookRepository.java

```
1  import com.itheima.chapter05.entity.Book;
2  import org.springframework.data.jpa.repository.JpaRepository;
3  import org.springframework.data.jpa.repository.Modifying;
4  import org.springframework.data.jpa.repository.Query;
5  import org.springframework.data.repository.query.Param;
6  import org.springframework.stereotype.Repository;
```

```
7  import org.springframework.transaction.annotation.Transactional;
8  import java.util.List;
9  @Repository
10 public interface BookRepository extends JpaRepository<Book,Integer> {
11     Book findByAuthorAndStatus(String author,String status);
12     @Modifying
13     @Transactional
14     @Query("delete from Book b where b.id = :id")
15     List<Book> deleteBookById(@Param("id") Integer id);
16 }
```

在上述代码中，第 9 行代码使用@Repository 注解标注接口，使项目启动时可以被 Spring 扫描到，交由 Spring 管理；第 10 行代码指定当前接口继承 JpaRepository 接口，并指定操作的实体类是 Book，其主键类型是 Integer；第 11 行代码使用 Spring Data 的方法命名规则定义查询方法 findByAuthorAndStatus()，用于根据图书作者和图书状态查询图书信息；第 12～15 行代码使用 JPQL 根据图书编号删除对应的图书信息，由于涉及对数据的 DML 操作，在第 12 行代码和第 13 行代码使用了@Modifying 注解和@Transactional 注解。

6．创建数据库

至此，实体类和操作实体的接口都已经定义好，因为没有在 JPA 的配置信息中设置项目启动时根据实体类自动创建数据表，此时测试对图书信息的操作需要先创建对应的数据库和数据表，并插入对应的测试数据，相关 SQL 如文件 5-5 所示。

文件 5-5　book.sql

```
CREATE DATABASE springbootdata;
USE springbootdata;
CREATE TABLE book (
  id int(0) NOT NULL AUTO_INCREMENT,
  name varchar(32) ,
  author varchar(32),
  press varchar(32),
  status varchar(1) ,
  PRIMARY KEY (id) USING BTREE
) ;
INSERT INTO book VALUES (1, '楚辞', '屈原', '中国文联出版社', '0');
INSERT INTO book VALUES (2, '纳兰词', '纳兰性德', '中国文联出版社', '1');
INSERT INTO book VALUES (3, '西游记', '吴承恩', '中国文联出版社', '2');
```

7．创建项目启动类和测试类

在 src\main\java 目录的 com.itheima.chapter05 包下创建项目启动类 Chapter05Application，具体如文件 5-6 所示。

文件 5-6　Chapter05Application.java

```
1  import org.springframework.boot.SpringApplication;
2  import org.springframework.boot.autoconfigure.SpringBootApplication;
3  @SpringBootApplication
4  public class Chapter05Application {
5      public static void main(String[] args) {
6          SpringApplication.run(Chapter05Application.class, args);
7      }
8  }
```

在 src\test\java 目录下创建包 com.itheima.chapter05，并在该包下创建测试类 Chapter05ApplicationTests。在 Chapter05ApplicationTests 测试类中定义操作图书信息的测试方法，具体如文件 5-7 所示。

文件 5-7 Chapter05ApplicationTests.java

```
import com.itheima.chapter05.dao.BookRepository;
import com.itheima.chapter05.entity.Book;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import java.util.List;
import java.util.Optional;
@SpringBootTest
class Chapter05ApplicationTests {
    @Autowired
    private BookRepository bookRepository;
    private  void booksInfo(){
        List<Book> books = bookRepository.findAll();
        for (Book book : books) {
            System.out.println(book);
        }
    }
    @Test
    void saveBook() {
        Book book=new Book(null,"离骚","屈原","清华大学出版社","1");
        //新增图书信息
        bookRepository.save(book);
        booksInfo();
    }
    @Test
    void editBook() {
        Optional<Book> op = bookRepository.findById(1);
        Book book = op.get();
        book.setName("天问");
        //修改图书信息
        bookRepository.save(book);
        booksInfo();
    }
    @Test
    void findBook() {
        //根据图书作者和图书状态查找对应的图书
        bookRepository.findByAuthorAndStatus("屈原","1");
       System.out.println(b);
    }
    @Test
    void delBook() {
        //根据图书编号删除图书信息
        bookRepository.deleteBookById(1);
        booksInfo();
    }
}
```

在上述代码中，第 12～45 行代码定义了 5 个方法。其中，第 12～17 行代码的 booksInfo()方法用于查询所有图书信息，并输出到控制台，第 13 行代码的 findAll()方法并没有在 BookRepository 接口中定义，而是继承自父接口。

第 19～24 行代码的 saveBook()方法用于测试新增图书信息，其中第 22 行代码调用的 save()方法并没有在 BookRepository 接口中定义，而是继承自父接口；新增图书后第 23 行代码调用 booksInfo()方法查询所有图书信息。

第 26～33 行代码的 editBook()方法用于测试修改图书信息，其中第 27 行代码调用父接口的 findById()方法获取对应的图书信息；第 31 行代码将修改后的图书信息传入 save()方法修改图书信息。

第 35～39 行代码的 findBook()方法中，调用 BookRepository 接口中自定义的 findByAuthorAndStatus()方法，根据图书作者和图书状态查找对应的图书。

第 41～45 行代码的 delBook()方法中，调用 BookRepository 接口中自定义的 deleteBookById()方法，根据图书编号删除对应的图书信息。

8. 测试操作图书信息

运行文件 5-7 中的 saveBook()方法，测试新增图书信息，控制台输出如图 5-6 所示。

图5-6　新增图书信息（1）

从图 5-6 可以看出，控制台输出了 4 条图书信息，其中第 4 条信息为新增的图书信息，说明执行 saveBook()方法后成功新增了图书信息。

运行文件 5-7 中的 editBook()方法，测试修改图书信息，控制台输出如图 5-7 所示。

图5-7　修改图书信息（1）

从图 5-7 可以看出，控制台输出了 4 条图书信息，其中第 1 条信息图书名称为“天问”，说明执行 editBook()方法后成功修改了图书编号为 1 的图书信息。

运行文件 5-7 中的 findBook()方法，测试查询图书信息，控制台输出如图 5-8 所示。

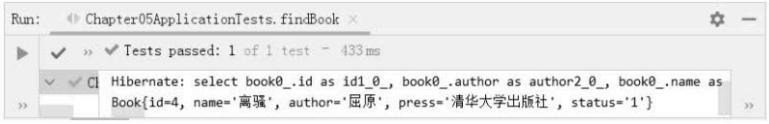

图5-8　查询图书信息（1）

从图 5-8 可以看出，控制台输出了 1 条图书信息，图书作者为屈原，图书状态为 1，说明执行 findBook()方法后成功查找到指定图书名称和图书状态的图书信息。

运行文件 5-7 中的 delBook()方法，测试删除图书信息，控制台输出如图 5-9 所示。

图5-9 删除图书信息（1）

从图 5-9 可以看出，控制台输出了图书编号为 2～4 的 3 条图书信息，说明执行 delBook() 方法后成功删除了图书编号为 1 的图书信息。

至此，Spring Boot 整合 Spring Data JPA 后，在项目中演示 Spring Data JPA 的基本使用已经完成。

5.3 Spring Boot 整合 MyBatis-Plus

MyBatis 是一个足够灵活的 DAO 层解决方案，但其也存在一些不足。为了弥补这些不足，可以对 MyBatis 现有的功能进行增强，MyBatis-Plus 就是这样的 MyBatis 增强工具。下面将对 MyBatis-Plus 的相关基础知识和 Spring Boot 整合 MyBatis-Plus 进行讲解。

5.3.1 MyBatis-Plus 概述

MyBatis 是半自动化的 ORM 实现，支持定制化 SQL、存储过程和高级映射，其封装性低于 Hibernate，但性能优秀、小巧、简单易学，在国内备受开发人员的喜爱。MyBatis 本身也存在些许不足，例如，配置文件繁多，以及当编写一个业务逻辑时需要在 DAO 层写一个方法，再创建一个与之对应的映射文件或 SQL 语句。针对 MyBatis 的这些不足，MyBatis-Plus 诞生了。MyBatis-Plus 是 MyBatis 的增强工具，在 MyBatis 的基础上只做增强不做改变，支持 MyBatis 所有原生的特性，旨在简化开发、提高效率。

MyBatis-Plus 除了弥补 MyBatis 的些许不足外，还具有以下特性。

- 无侵入：只做增强不做改变，引入它不会对现有工程产生影响。
- 损耗小：启动即会自动注入基本增删改查方法，性能基本无损耗，直接面向对象操作。
- 强大的增删改查操作：内置通用 Mapper、通用 Service，仅仅通过少量配置即可实现单表大部分增删改查操作，具有强大的条件构造器，可满足各类使用需求。
- 支持 Lambda 形式调用：通过 Lambda 表达式，可方便地编写各类查询条件，无须再担心字段写错。
- 支持主键自动生成：支持分布式唯一 ID 生成器 Sequence 在内的 4 种主键策略，可自由配置，完美解决了主键问题。
- 支持 ActiveRecord 模式：支持 ActiveRecord 形式调用，实体类只需继承 Model 类即可进行强大的增删改查操作。
- 支持自定义全局通用操作：支持全局通用方法注入。
- 内置代码生成器：采用代码或者 Maven 插件可快速生成 Mapper 、Model、Service、Controller 层代码，支持模板引擎。
- 内置分页插件：基于 MyBatis 物理分页，开发者无须关心具体操作，配置好插件之后，写分页等同于普通 List 查询。
- 分页插件支持多种数据库：支持 MySQL、MariaDB、Oracle、DB2、H2、HSQL、SQLite、PostgreSQL、SQLServer 等多种数据库。

- 内置性能分析插件：可输出 SQL 语句以及其执行时间，建议开发测试时启用该功能，能快速揪出慢查询。
- 内置全局拦截插件：提供全表 delete、update 操作智能分析阻断，也可自定义拦截规则，预防误操作。

MyBatis-Plus 提供了一系列简化数据库操作的模块和功能，主要包含注解模块（annotation）、拓展功能（extension）、核心模块（core）、代码生成器（generator），以及启动器（mybatis-plus-boot-starter），MyBatis-Plus 的框架结构如图 5-10 所示。

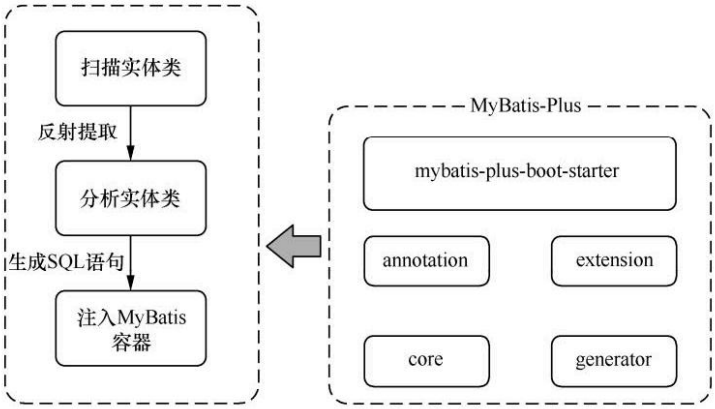

图5-10　MyBatis-Plus的框架结构

从图 5-10 可以得出，使用 MyBatis-Plus 时，MyBatis-Plus 首先会扫描指定的包路径下所有实体类，并根据实体类中的注解和属性，分析出其对应的数据表；接着 MyBatis-Plus 会根据实体类的属性，分析并生成出对应的 SQL 语句；然后 MyBatis-Plus 会将生成的 SQL 语句注入 MyBatis 的容器中；最后开发者通过调用 MyBatis-Plus 提供的接口，执行对应的 SQL 语句，实现对数据库的操作。

5.3.2　MyBatis-Plus 快速入门

通过学习 MyBatis-Plus 的特性可知，MyBatis-Plus 内置了通用 Mapper 和通用 Service，启动时会自动注入基本的增删改查方法，并提供了强大的条件构造器，下面对 MyBatis-Plus 的这些基本功能进行讲解。

1. 通用 Mapper

MyBatis-Plus 的通用 Mapper 是指其 BaseMapper 接口，MyBatis-Plus 在启动时会自动解析实体类和数据表的关系映射，同时根据实体类中的注解和属性生成对应的 SQL 语句，并将生成对应的 SQL 语句转换为 MyBatis 内部对象注入 Mybatis 容器中。BaseMapper 接口中封装了基本的增删改查方法，下面对这些方法分别进行讲解。

（1）插入方法

```
int insert(T entity);
```

insert()方法用于向数据库中插入一条记录，entity 为插入的实体对象。

（2）更新方法

```
// 根据 whereWrapper 条件，更新记录
int update(@Param(Constants.ENTITY) T updateEntity,
 @Param(Constants.WRAPPER) Wrapper<T> whereWrapper);
```

```
// 根据 ID 更新
int updateById(@Param(Constants.ENTITY) T entity);
```

上述方法为 BaseMapper 接口提供的两个更新方法，分别为根据指定的条件进行更新，以及根据实体对象的 ID 进行更新。其中，Wrapper 为条件构造器，其对象中封装了实体对象的信息，具体的使用会在后续章节详细讲解。

（3）删除方法

```
1  // 根据 entity 条件，删除记录
2  int delete(@Param(Constants.WRAPPER) Wrapper<T> wrapper);
3  // 根据 ID 批量删除
4  int deleteBatchIds(@Param(Constants.COLLECTION)
5      Collection<? extends Serializable> idList);
6  // 根据 ID 删除
7  int deleteById(Serializable id);
8  // 根据 columnMap 条件，删除记录
9  int deleteByMap(@Param(Constants.COLUMN_MAP)
10     Map<String, Object> columnMap);
```

上述方法为 BaseMapper 接口提供的 4 个删除方法，其中，第 4～5 行代码的 delete BatchIds()方法中 idList 为主键 ID 列表，不能为 null 和 empty；第 9～10 行代码的 deleteByMap()方法中 columnMap 为表字段对应的 Map 对象。

（4）查询方法

```
1  // 根据 id 查询
2  T selectById(Serializable id);
3  // 根据 entity 条件，查询一条记录
4  T selectOne(@Param(Constants.WRAPPER) Wrapper<T> queryWrapper);
5  // 根据 id 批量查询
6  List<T> selectBatchIds(@Param(Constants.COLLECTION)
7          Collection<? extends Serializable> idList);
8  // 根据 entity 条件，查询全部记录
9  List<T> selectList(@Param(Constants.WRAPPER) Wrapper<T> queryWrapper);
10 // 根据 columnMap 条件进行查询
11 List<T> selectByMap(@Param(Constants.COLUMN_MAP)
12         Map<String, Object> columnMap);
13 // 根据 Wrapper 条件，查询全部记录
14 List<Map<String, Object>> selectMaps(@Param(Constants.WRAPPER)
15         Wrapper<T> queryWrapper);
16 // 根据 Wrapper 条件，查询全部记录。注意： 只返回第一个字段的值
17 List<Object> selectObjs(@Param(Constants.WRAPPER)
18         Wrapper<T> queryWrapper);
19 // 根据 entity 条件，查询全部记录并翻页
20 IPage<T> selectPage(IPage<T> page,
21        @Param(Constants.WRAPPER) Wrapper<T> queryWrapper);
22 // 根据 Wrapper 条件，查询全部记录并翻页
23 IPage<Map<String, Object>> selectMapsPage(IPage<T> page,
24        @Param(Constants.WRAPPER) Wrapper<T> queryWrapper);
25 // 根据 Wrapper 条件，查询总记录数
26 Integer selectCount(@Param(Constants.WRAPPER) Wrapper<T> queryWrapper);
```

上述方法为 BaseMapper 接口提供的查询方法，其中，第 6～7 行代码的 selectBatchIds()方法中 idList 为主键 id 列表，不能为 null 和 empty；第 11～12 行代码的 selectByMap()方法

中 columnMap 为表字段对应的 Map 对象；第 20～24 行代码的 selectPage()方法和 selectMaps Page()方法中 page 为分页查询条件。

自定义接口实现 BaseMapper 接口后，不需要手动实现该接口中的方法，也不需要编写对应的动态 SQL，MyBatis-Plus 会自动为 BaseMapper 中的接口类实现代理，可以直接使用其中的增删改查方法。

2. 通用 Service

除了通用 Mapper，MyBatis-Plus 还提供了通用 Service，通用 Service 是指其 IService 接口，该接口中也提供了基本的增删改查方法。编写 Service 层代码时，可以使用自定义的 Service 接口继承 IService 接口，调用 IService 接口中的方法时，不需要手动实现，即可实现基本的增删改查功能。下面对 IService 接口的常用方法进行讲解。

（1）插入方法

```
1 // 插入一条记录
2 boolean save(T entity);
3 // 批量插入
4 boolean saveBatch(Collection<T> entityList);
5 // 批量插入
6 boolean saveBatch(Collection<T> entityList, int batchSize);
```

在上述方法中，entity 是插入的实体对象；entityList 是实体对象集合；batchSize 是插入批次数量。

（2）更新方法

```
1 // 根据 UpdateWrapper 条件，更新记录需要设置 sqlset
2 boolean update(Wrapper<T> updateWrapper);
3 // 根据 whereWrapper 条件，更新记录
4 boolean update(T updateEntity, Wrapper<T> whereWrapper);
5 // 根据 ID 选择修改
6 boolean updateById(T entity);
7 // 根据 ID 批量更新
8 boolean updateBatchById(Collection<T> entityList);
9 // 根据 ID 批量更新
10 boolean updateBatchById(Collection<T> entityList, int batchSize);
```

在上述方法中，updateWrapper 是条件构造器；entityList 是实体对象集合；batchSize 是更新批次数量。

（3）插入或更新方法

```
1 // 主键存在就更新记录，否则插入一条记录
2 boolean saveOrUpdate(T entity);
3 // 根据 updateWrapper 进行插入或更新
4 boolean saveOrUpdate(T entity, Wrapper<T> updateWrapper);
5 // 批量插入或更新
6 boolean saveOrUpdateBatch(Collection<T> entityList);
7 // 批量插入或更新
8 boolean saveOrUpdateBatch(Collection<T> entityList, int batchSize);
```

在上述方法中，当插入或更新时，会先判断操作的 entity 的主键在数据库中是否存在对应的记录，如果记录存在则选择修改，如果记录不存在则选择插入。

（4）删除方法

```
1 // 根据 entity 条件，删除记录
```

```
boolean remove(Wrapper<T> queryWrapper);
// 根据 id 删除
boolean removeById(Serializable id);
// 根据 columnMap 条件，删除记录
boolean removeByMap(Map<String, Object> columnMap);
// 根据 id 批量删除
boolean removeByIds(Collection<? extends Serializable> idList);
```

在上述方法中，queryWrapper 为条件构造器；id 为删除记录对应的 id；columnMap 为表字段对应的 Map 对象；idList 为批量删除时对应记录的主键列表。

（5）查询方法

```
// 根据 ID 查询
T getById(Serializable id);
// 根据 Wrapper，查询一条记录。结果集，如果是多个会抛出异常
T getOne(Wrapper<T> queryWrapper);
// 根据 Wrapper，查询一条记录
T getOne(Wrapper<T> queryWrapper, boolean throwEx);
// 根据 Wrapper，查询一条记录
Map<String, Object> getMap(Wrapper<T> queryWrapper);
// 根据 Wrapper，查询一条记录
<V> V getObj(Wrapper<T> queryWrapper, Function<? super Object, V> mapper);
// 查询所有
List<T> list();
// 查询列表
List<T> list(Wrapper<T> queryWrapper);
// 查询（根据 ID 批量查询）
Collection<T> listByIds(Collection<? extends Serializable> idList);
// 查询（根据 columnMap 条件）
Collection<T> listByMap(Map<String, Object> columnMap);
// 查询所有列表
List<Map<String, Object>> listMaps();
// 查询列表
List<Map<String, Object>> listMaps(Wrapper<T> queryWrapper);
// 查询全部记录
List<Object> listObjs();
// 查询全部记录
<V> List<V> listObjs(Function<? super Object, V> mapper);
// 根据 Wrapper 条件，查询全部记录
List<Object> listObjs(Wrapper<T> queryWrapper);
```

在上述方法中，第 2～10 行都是查询单条记录的方法；第 12～28 行都是查询一条及以上记录的方法。

（6）分页查询方法

```
// 无条件分页查询
IPage<T> page(IPage<T> page);
// 条件分页查询
IPage<T> page(IPage<T> page, Wrapper<T> queryWrapper);
// 无条件分页查询
IPage<Map<String, Object>> pageMaps(IPage<T> page);
// 条件分页查询
IPage<Map<String, Object>> pageMaps(IPage<T> page,
```

```
9       Wrapper<T> queryWrapper);
```

在上述代码中，page 为翻页对象。

MyBatis-Plus 中提供 IService 接口用于封装业务逻辑层的增删改查方法，并且提供了 IService 接口对应的实现类 ServiceImpl，ServiceImpl 中注入了 BaseMapper 接口的实例，执行 ServiceImpl 实现类的方法时，会自动通过注入 BaseMapper 接口的实例实现与数据库的交互。因此，开发人员在业务逻辑层只需继承 ServiceImpl 即可使用通用 Service 中的方法。

3. 条件构造器

在开发中，有时候希望查询根据所指定的条件进行增删改查操作，而这个条件可能包含多种情况。在 MyBatis-Plus 中可以使用条件构造器 Wrapper 根据具体的需求定义来封装指定的条件，下面对条件构造器的常用方法进行讲解。

（1）eq

```
eq(R column, Object val)
```

在上述代码中，column 表示数据库字段名；val 表示字段值。eq()方法用于匹配字段中值等于某个值的记录，例如，eq("name", "老王")用于匹配 name 字段中值等于"老王"的记录。

（2）ne

```
ne(R column, Object val)
```

ne()方法用于匹配字段中值不等于某个值的记录，例如，eq("name", "老王")用于匹配 name 字段中值不等于"老王"的记录。

（3）gt

```
gt(R column, Object val)
```

gt()方法用于匹配字段中值大于某个值的记录，例如， gt("age", 18)用于匹配 age 字段中值大于 18 的记录。

（4）ge

```
ge(R column, Object val)
```

ge()方法用于匹配字段中值大于或等于某个值的记录，例如， ge("age", 18)用于匹配 age 字段中值大于或等于 18 的记录。

（5）lt

```
lt(R column, Object val)
```

lt()方法用于匹配字段中值小于某个值的记录，例如， lt("age", 18)用于匹配 age 字段中值小于 18 的记录。

（6）le

```
le(R column, Object val)
```

le()方法用于匹配字段中值小于或等于某个值的记录，例如， le("age", 18)用于匹配 age 字段中值小于或等于 18 的记录。

（7）between

```
between(R column, Object val1, Object val2)
```

between()方法用于匹配字段中值在指定区间的记录，例如， between("age", 18, 30)用于匹配 age 字段中值大于 18 并且小于 30 的记录。

（8）like

```
like(R column, Object val)
```

like()方法用于模糊匹配字段中的值，例如，like("name", "王")用于匹配 name 字段中值